

Go runtime internals

Goroutines • G/M/P • scheduler • work stealing • GOMAXPROCS
netpoller • syscalls • stack growth • стоимость goroutine

Главный вопрос

На машине 8 CPU, а приложение создаёт 100 000 goroutine. Кто их выполняет?

Почему Go не создаёт 100 000 OS threads?

Что происходит, когда goroutine ждёт сеть, mutex или syscall?

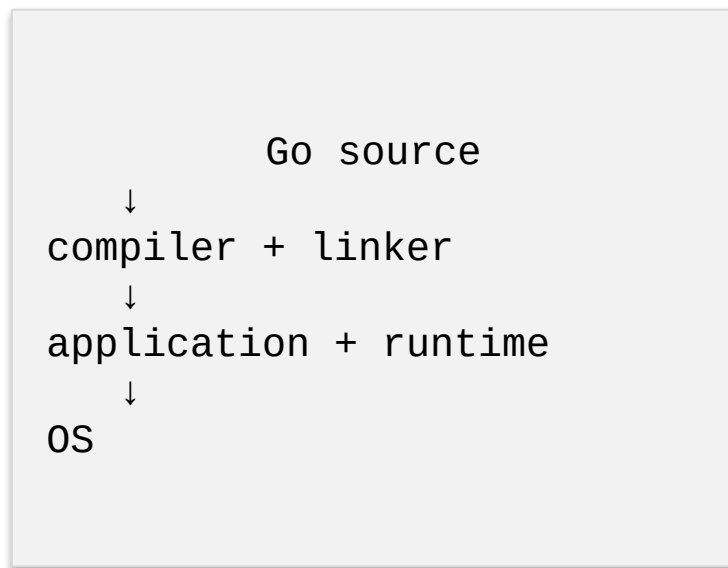
Ответ: Go runtime multiplexes множество goroutine поверх ограниченного набора OS threads.

Go runtime — часть executable

Go компилируется в native executable, но это не «просто машинный код без runtime».

Runtime содержит scheduler, GC, allocator, stack management, timers, netpoller и служебную metadata.

В отличие от JVM runtime не запускается отдельным процессом — он линкован с приложением.



Goroutine: точная ментальная модель

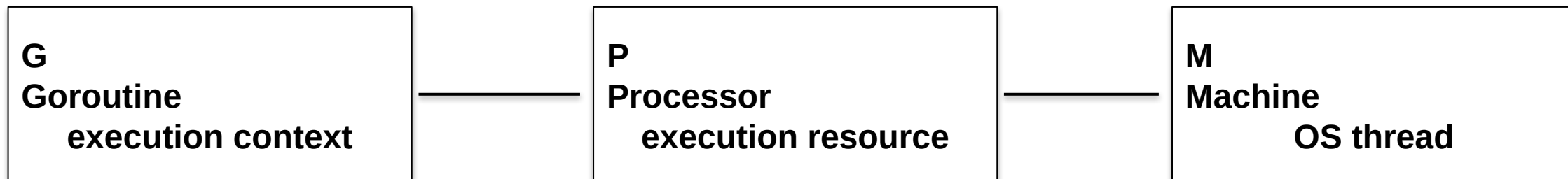
Упрощение: goroutine = «лёгкий thread».

Точнее: goroutine — independently schedulable execution context, которым управляет Go runtime.

Она имеет собственный user stack и scheduler/runtime state.

``go f()`` делает новую goroutine runnable; когда она реально получит CPU, решает scheduler.

G / M / P — три сущности scheduler



G хранит состояние конкретной goroutine.

M физически исполняет инструкции как OS thread.

P нужен M для исполнения Go-кода и несёт per-P runtime resources.

Жизненный цикл G



Большинство из 100 000 goroutine обычно не **RUNNING**: они ждут I/O, timer, channel, mutex или другой ресурс.

M — настоящий OS thread

M создаётся/управляется runtime, но планируется уже kernel scheduler.

Именно M исполняет machine instructions.

Количество M может быть больше GOMAXPROCS: часть M может блокироваться в syscalls/cgo.

Go scheduler выбирает G для M; OS scheduler выбирает, когда сам M получит CPU.

Зачем вообще нужен P?

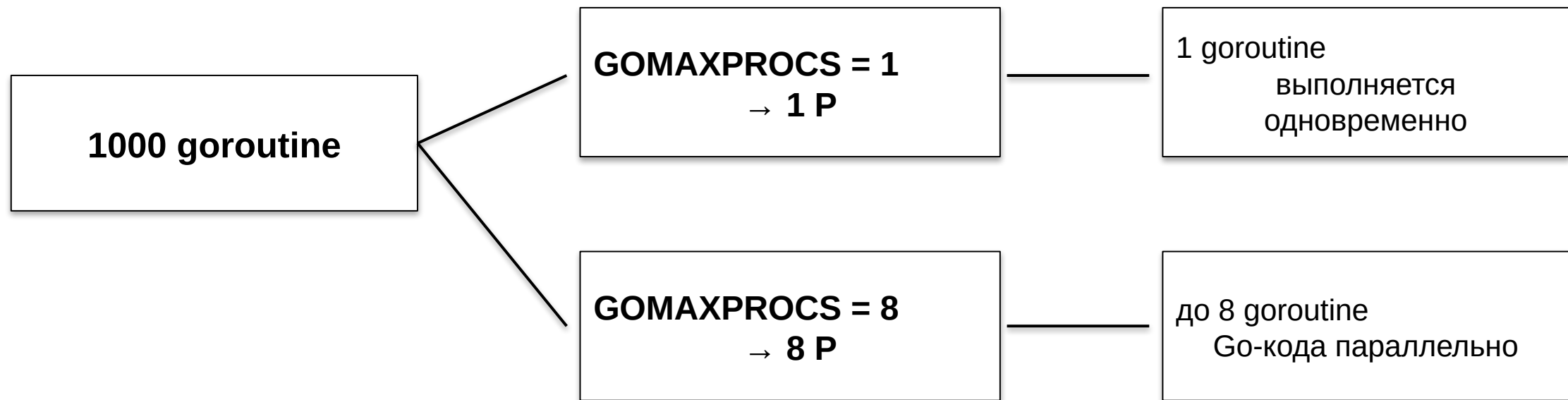
Модель $G \rightarrow M$ недостаточна: runtime нужны per-execution ресурсы, которые нельзя навечно привязать к OS thread.

P содержит локальную run queue и другую per-P runtime state.

M может заблокироваться в syscall и временно потерять P.

Освободившийся P может подхватить другой M и продолжить выполнение Go-кода.

Concurrency ≠ parallelism



GOMAXPROCS

Практическая модель: $GOMAXPROCS \approx \text{количество } P \approx \text{максимальный parallelism Go-кода.}$

Это не количество goroutine и не верхняя граница OS threads процесса.

CPU-bound workload ограничен количеством доступных P/CPU.

I/O-heavy workload может иметь огромное число goroutine при небольшом числе одновременно RUNNING.

GOMAXPROCS в контейнере

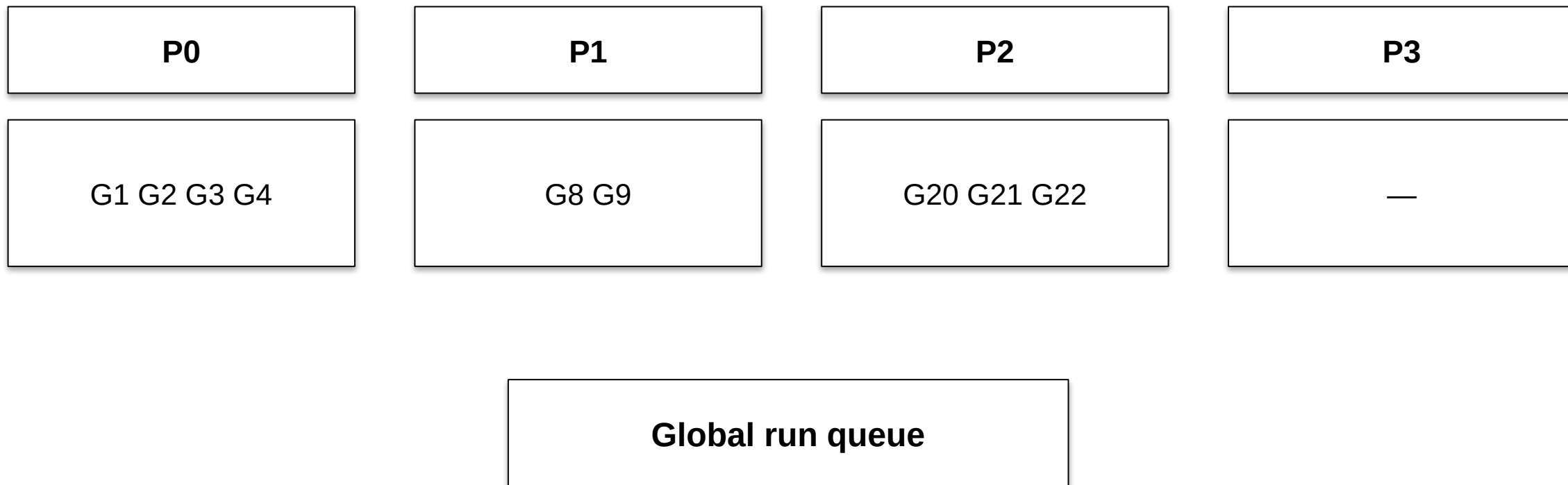
Проблема: host может иметь 128 CPU, а cgroup/container limit — 2 CPU.

Слишком высокий parallelism приводит к quota exhaustion → kernel throttling → ухудшению tail latency.

Современный Go учитывает container CPU limits при выборе default GOMAXPROCS.

Цепочка для backend: GOMAXPROCS → P → M → Linux scheduler → cgroups → p99 latency.

Где ждут RUNNABLE goroutine



Local queues уменьшают contention. Global queue нужна для координации/fairness и для работы, не привязанной к одному P.

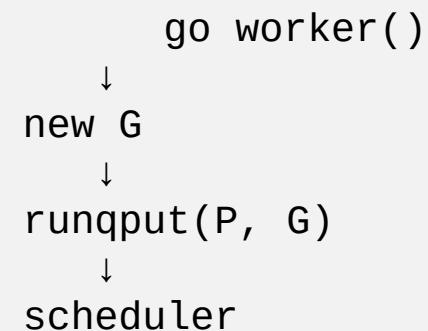
Что делает `go f()`

Compiler генерирует обращение к runtime machinery создания goroutine.

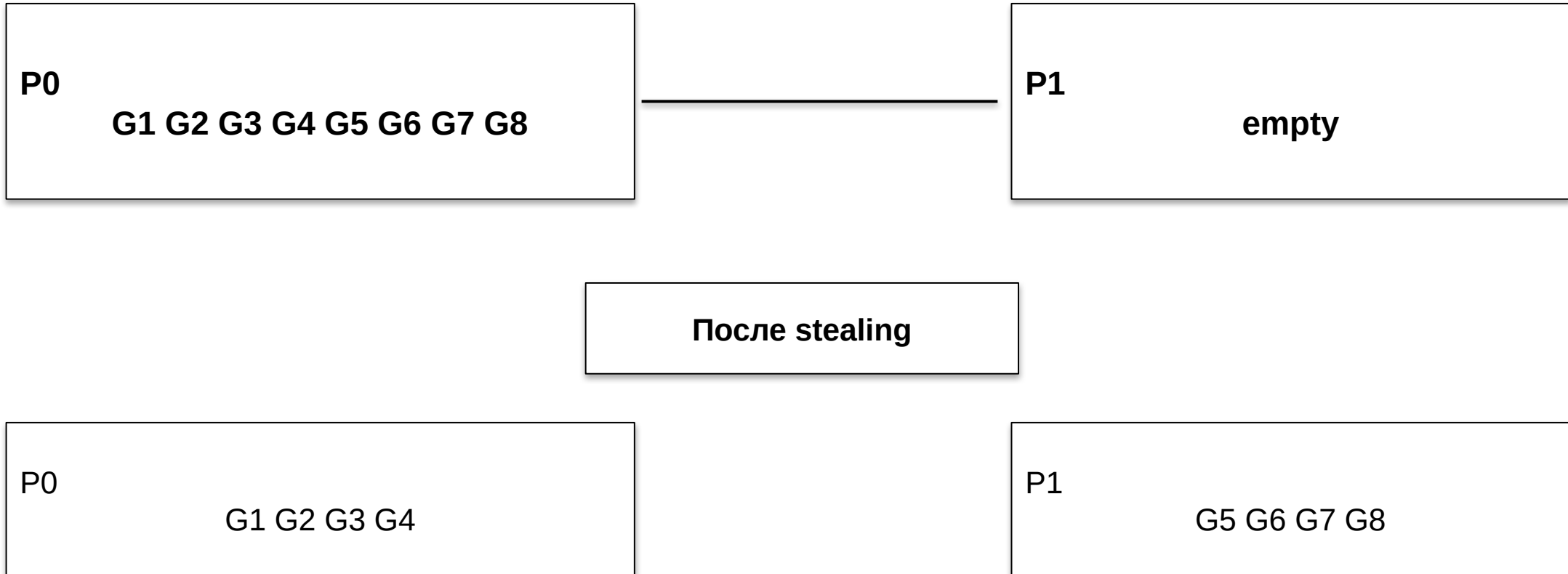
Runtime создаёт новую G и помещает её в runnable queue текущего P.

В текущей реализации локальная очередь P имеет 256 slots плюс список ожидания.

Создание goroutine не означает немедленного запуска функции.



Work stealing: если одному P нечего делать



Почему воровать пачку работы

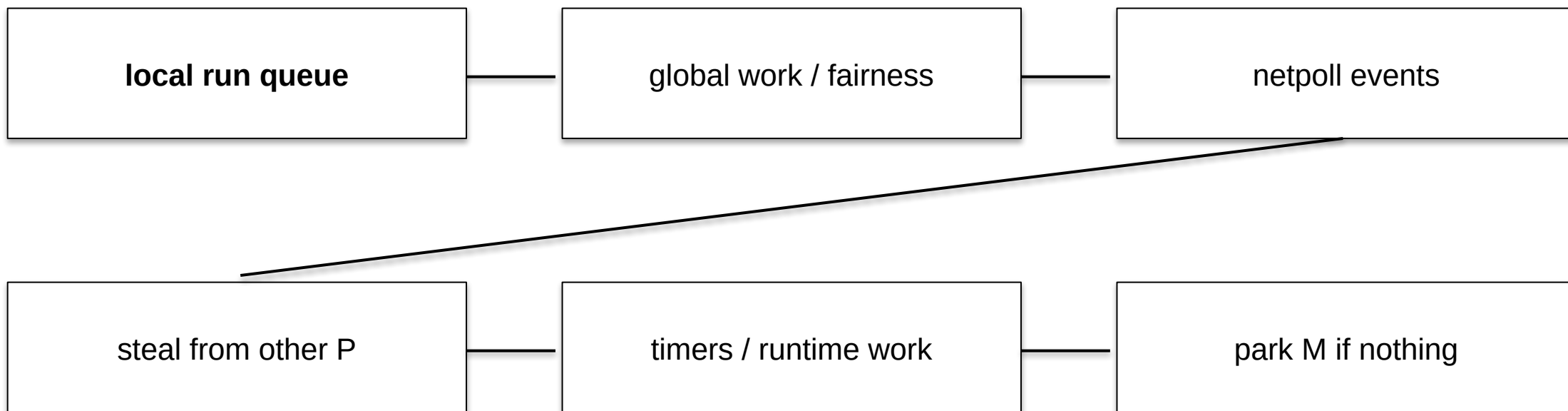
Украсть одну G: быстро закончили → снова идём искать работу → scheduler overhead.

Украсть всё: исходный P сам остаётся пустым.

Классический компромисс work stealing — забрать примерно половину runnable work.

Цель — балансировка без центрального scheduler-босса и глобального lock на каждую операцию.

Упрощённый цикл scheduler



Блокировка goroutine ≠ блокировка OS thread

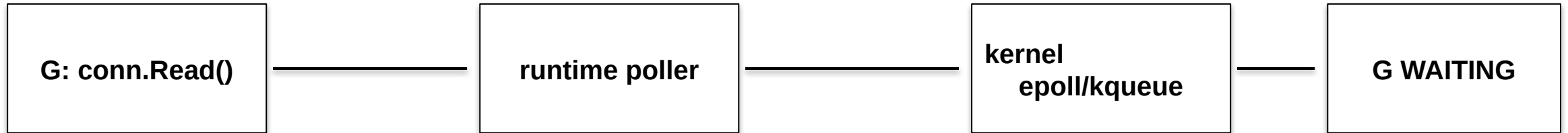
Channel receive, mutex, timer и другие runtime-aware waits могут перевести G в WAITING.

М при этом может продолжить выполнять другую runnable G.

Scheduler сохраняет execution context G и возвращает её в runnable state, когда событие произошло.

Именно это позволяет тысячам ожидающих goroutine не требовать тысячи running threads.

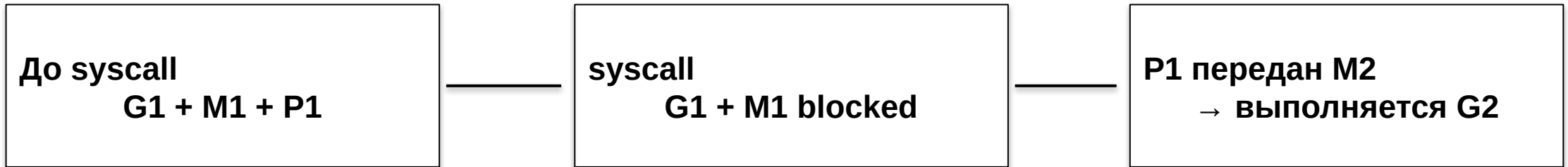
Netpoller: network I/O без thread-per-connection



packet/data ready → G becomes RUNNABLE → scheduler executes it again

Код выглядит синхронным, но ожидание pollable socket интегрировано с scheduler.

Blocking syscall: M действительно может застрять в kernel



После возврата M1 должен снова получить P; если свободного P нет — G1 продолжит позже через scheduler.

Netpoller и blocking syscall — разные механизмы

Pollable network I/O

G ждёт
M свободен для другой G
OS thread не обязан блокироваться

Blocking syscall

G + M могут реально блокироваться в kernel
P отделяется
другой M продолжает Go-код

Всегда задавайте вопрос: goroutine блокируется НА ЧЁМ? От этого зависит runtime-механика.

Preemption: что если G не блокируется добровольно

CPU-bound tight loop не должен бесконечно монополизировать P.

Современный runtime поддерживает asynchronous preemption (начиная с Go 1.14).

В текущей реализации есть внутренний ориентир порядка 10 ms для force-preemption, но это не гарантированный quantum.

Scheduling остаётся best-effort и зависит также от OS scheduler, GC и нагрузки.

Stack goroutine: маленький и динамический



Нет места → allocate larger contiguous stack → copy used frames → fix pointers → continue

Современный runtime может адаптивно выбирать стартовый stack выше минимума по статистике workload.

Почему stack growth — сложнее обычного memсru

После переноса stack его адрес меняется.

Pointers, направленные внутрь старого stack, должны быть скорректированы.

Compiler-generated stack maps помогают runtime понимать pointer layout frames.

Особые runtime structures (defer/panic/channel wait state и др.) тоже требуют корректировки.

У М есть системный stack — g0

User goroutine исполняется на собственном growable stack.

Runtime scheduler и часть внутренних операций выполняются на system stack М (g0).

Это позволяет runtime выполнять операции, которые сами не должны зависеть от обычного user-stack growth/preemption.

Модель: М переключается между user G и своей служебной g0.

Сколько стоит goroutine

Goroutine намного дешевле традиционного OS thread для большого числа waiting tasks, но она не бесплатна.

Цена: G metadata + stack + scheduler work + возможные allocations/captures + GC scanning.

$100\,000 \times 2\text{ KiB} \approx 195\text{ MiB}$ только как грубая нижняя оценка stack memory.

Реальный объём может быть значительно больше из-за heap objects, buffers, timers, sockets и более крупных stacks.

Goroutine leak: scheduler работает правильно, архитектура — нет

```
func process() {  
    ch := make(chan Result)  
  
    go func() {  
        ch <- slowOperation()  
    }()  
  
    return // никто не читает ch  
}
```

1 request → +1 leaked G
1000 requests → +1000 G

Симптомы:

goroutines ↑

memory ↑

GC CPU ↑

latency ↑

Scheduler не создаёт backpressure

Миллион дешёвых goroutine перед PostgreSQL с pool size 20 — всё ещё плохая система.

Scheduler знает, как распределять runnable work; он не знает capacity вашей БД или downstream API.

Нужны bounded concurrency, semaphore, worker pool, connection pool, queue limits или rate limiter.

Дешёвая goroutine — очень дешёвый способ случайно создать огромную очередь прямо в памяти процесса.

Диагностика runtime/scheduler

`runtime.NumGoroutine()` / runtime metrics — динамика числа G.

`pprof` goroutine profile — где и на чём ждут goroutine.

`go tool trace` — lifecycle G, scheduler latency, network wait, syscalls, GC, P/M activity.

`GODEBUG=schedtrace=1000,scheddetail=1` — snapshot scheduler state.

Ключевой вопрос: CPU bottleneck, network wait, syscall, mutex/channel, connection pool или leak?

Java-мост: Thread / Virtual Thread / Goroutine

Java platform Thread
≈ OS thread

Java virtual thread
user-mode task → carrier
thread

Go goroutine
G → Go scheduler → M

Общая идея: дешёвый logical execution context не делает CPU, DB connections и downstream capacity бесконечными.

Финальная модель

G G G G G G G G

↓
local/global runnable queues

P0 P1 P2 P3

↓ ↓ ↓ ↓
M0 M1 M2 M3

↓
OS scheduler

↓
CPU

network: G → netpoller → WAITING → RUNNABLE

syscall: G+M → kernel, P → another M

stack: no space → grow/copy/fix → continue

Что важно запомнить

$G \neq \text{OS thread}$. G — runtime execution context; M — OS thread; P — ресурс исполнения Go-кода.

GOMAXPROCS ограничивает parallelism Go-кода через число P , но не число goroutine или M .

Local run queues + work stealing уменьшают contention и балансируют runnable work.

Netpoller позволяет тысячам сетевых goroutine ждать I/O без thread-per-connection.

Blocking syscall может удерживать M , поэтому P отделяется и продолжает работу с другим M .

Growable stacks уменьшают стартовую стоимость G , но goroutine всё равно потребляет память и GC/scheduler resources.

Scheduler решает scheduling, но не backpressure и не lifecycle ваших ресурсов.

Go scheduler deep dive

Путь одной goroutine через runtime/proc.go

```
newproc → runqput → schedule → findRunnable → execute  
→ gopark / goready → runqsteal
```

Главный вопрос

Что реально происходит после `go worker()`?

- Кто создаёт `G` и куда она попадает?
- Как `scheduler` выбирает следующую goroutine?
- Что происходит при блокировке и пробуждении?
- Как один `P` забирает работу у другого?

Полный pipeline новой goroutine

```
go worker()
```

↓

```
newproc → newproc1 → _Grunnable → runqput
```

↓

```
schedule → findRunnable → execute
```

↓

```
_Grunning → worker()
```

Создание и запуск — разные фазы: новая G сначала становится runnable.

go f() ≠ немедленный запуск

```
func main() {  
  go worker()  
}
```

Компилятор подводит вызов к runtime.newproc.

Runtime создаёт/переиспользует G, переводит её в runnable и кладёт в очередь.

Когда она реально получит CPU — решит scheduler.

Почему runtime использует system stack

user G stack

↓ systemstack

M.g0 system stack

↓

создание G / scheduler metadata / run queue operations

Низкоуровневые операции scheduler выполняются на системном stack M.g0.

newproc1: G можно переиспользовать

```
go worker()
```

```
↓
```

```
gfget(P): есть свободная G?
```

```
├ yes → reuse G
```

```
└ no  → allocate G + stack
```

```
↓
```

```
prepare execution context
```

```
↓
```

```
status = _Grunnable
```

Завершившиеся G runtime держит в пуле и переиспользует.

P: runnext + local run queue

P0

└─ runnext → G42

└─ runq[256] → G10 G11 G12 ...

runnext – кандидат на быстрый следующий запуск.

runq – локальная кольцевая очередь runnable goroutines.

Локальная очередь уменьшает глобальный contention.

Зачем существует `runnext`

Parent создаёт child:

```
go child()
```

Если child отправить в хвост очереди из сотен G – растёт `scheduling latency`.

`runnext` позволяет связанным goroutines быстрее передавать выполнение.

Это `latency optimization`, а не строгий `priority mechanism`.

Переполнение local runq

P0 local runq: ~256 G

+ новая G

↓

runqputslow

↓

часть (~половина) остаётся локально

часть batch-ем уходит в global queue

Так перегруженный P разгружается, а работа становится доступна другим P.

waker: почему нельзя будить thread на каждую G

Каждый park/unpark OS thread имеет цену.

Если будить M на каждую новую G:

work появилась → wake thread → work уже забрал другой M → thread снова sleep.

Runtime будит worker консервативно: нужен idle P и не должно быть уже достаточного spinning M.

Spinning M

SPINNING M

↓

активно ищет работу:

local / global / netpoll / steal

↓

не нашёл

↓

PARK

Сразу park → дорогие wakeups.

Вечно spin → пустая трата CPU.

Spinning — компромисс между latency и CPU.

schedule(): один раунд scheduler

```
schedule()
```

```
↓
```

```
findRunnable()
```

```
↓
```

```
execute(gp, inheritTime)
```

Смысл `schedule` прост: найти runnable `G` и передать ей исполнение.

Большая часть сложности спрятана в поиске работы и безопасных переходах состояний.

findRunnable(): работа не в одной очереди

findRunnable() ищет среди:

1. runtime / GC / timers
2. periodic global fairness check
3. local runq
4. global runq
5. non-blocking netpoll
6. work stealing
7. blocking netpoll / park

Точный порядок — implementation detail runtime.

Locality vs fairness

Local runq:

- + минимум shared coordination
- + cache locality

Но бесконечное предпочтение local work может голодать global queue.

Поэтому runtime периодически проверяет global work раньше local.

Scheduler постоянно балансирует locality и fairness.

Global queue забирается batch-ем

GLOBAL QUEUE

G1 G2 G3 G4 G5 ...

↓ take batch

P2 получает:

- одну G для немедленного execute
- остальные — в local runq

Один доступ к global queue амортизируется: дальше P снова работает локально.

Netpoller перед тяжёлым stealing

Local empty. Global empty.

Kernel мог уже сделать socket readable/writable.

Неблокирующий netpoll возвращает G, которые стали runnable.

Получить готовую I/O goroutine часто дешевле, чем обходить другие P в поиске чужих очередей.

Work stealing

До:

P0: G1 G2 G3 G4 G5 G6 G7 G8

P1: —

P1 идёт steal у P0

После примерно:

P0: G1 G2 G3 G4

P1: G5 G6 G7 + одна G в execute

Пустой P не должен простаивать, пока где-то есть runnable work.

Почему крадём примерно половину

Украсть одну G:

→ быстро снова придётся steal

→ scheduler overhead

Украсть всё:

→ victim сам останется без work

Половина — компромисс: thief получает запас работы, victim сохраняет locality и progress.

execute(): G получает M

```
findRunnable вернул gp
↓
execute(gp)
↓
M.curg = G
G.m = M
_Grunnable → _Grunning
↓
gogo(&G.sched)
↓
восстановить PC/SP/register context
↓
продолжить user code
```

Scheduler не интерпретирует каждую инструкцию

После `gogo` CPU просто исполняет `user code`.

Scheduler возвращается в игру, когда возникает событие:

- `blocking`
- `syscall`
- `preemption`
- `Gosched`
- `stack growth`
- `panic / return`

Scheduler — `event-driven machinery`, а не `interpreter`.

gopark: goroutine больше не может работать

G42 RUNNING

↓

gopark

↓

switch to g0 / park_m

↓

_Grunning → _Gwaiting

↓

detach G from M

↓

schedule another G

Block G ≠ block OS thread для runtime-aware primitives.

goready: событие произошло

event happened

↓

goready(G42)

↓

ready()

↓

_Gwaiting → _Grunnable

↓

runqput

↓

wakeup if needed

Важно: goready не запускает G немедленно — она снова только runnable.

Channel wait: полный lifecycle

```
G1 RUNNING
↓ receive from empty channel
register waiter
↓
gopark → G1 WAITING
↓
M/P executes G2
↓
G2 sends value
↓
goready(G1) → RUNNABLE
↓
scheduler → execute
↓
G1 RUNNING, receive returns
```

Почему park и waiter должны быть согласованы

Опасная гонка:

G1: check → empty

[context switch]

G2: send → no waiter → done

G1: park forever

Это lost wakeup.

Blocking primitive должен согласовать регистрацию waiter и переход в sleep так, чтобы событие невозможно было потерять.

Gosched / preemption $\neq$ park

gopark:

RUNNING $\rightarrow$ WAITING

G сейчас НЕ МОЖЕТ продолжать

Gosched / preemption:

RUNNING $\rightarrow$ RUNNABLE

G МОЖЕТ работать, но уступает execution resource

Это разные причины scheduler switch.

Завершение goroutine

worker() returns

↓

goexit / gdestroy

↓

_Grunning → _Gdead

↓

G может попасть в pool

↓

позже reuse через gfget

Lifecycle объекта G длиннее lifecycle одного вызова goroutine.

Полный lifecycle одной G

```
go f() → newproc → newproc1
↓
RUNNABLE → runqput
↓
schedule → findRunnable → execute
↓
RUNNING
↓ block
gopark → WAITING
↓ event
goready → RUNNABLE
↓
execute → RUNNING
↓ return
goexit → DEAD → pool
```

Что происходит под backend load

100 000 HTTP goroutines $\neq$ 100 000 running tasks.

Например:

20 RUNNING

500 RUNNABLE

70 000 waiting network

20 000 waiting DB pool

9 480 waiting timer/channel/mutex

Диагностика начинается с состояния и wait reason, а не с самого числа goroutines.

Scheduler latency

G стала RUNNABLE в t_0

execute произошёл в $t_0 + 200\text{ ms}$

Это scheduler delay: G готова работать, но execution resource не получила.

Причины:

- CPU saturation
- огромные runnable queues
- GOMAXPROCS / cgroup throttling
- GC/runtime work

Не путать с WAITING на network/DB.

Диагностика: go tool trace

Trace позволяет увидеть:

G created → runnable → running → blocked → runnable → running

Смотрим:

- scheduler latency
- blocking reasons
- network wait
- syscalls
- GC
- P/M activity

Вопрос: handler медленный потому, что работает долго или потому, что долго ждёт CPU?

Что важно запомнить

- go f() создаёт runnable G, а не немедленный thread
- P держит local runq и runnext
- schedule = findRunnable → execute
- local queues дают locality, global queue — coordination/fairness
- work stealing балансирует P
- gopark: RUNNING → WAITING
- goready: WAITING → RUNNABLE
- scheduler управляет execution, но не заменяет backpressure
- под нагрузкой важны RUNNING / RUNNABLE / WAITING и wait reason

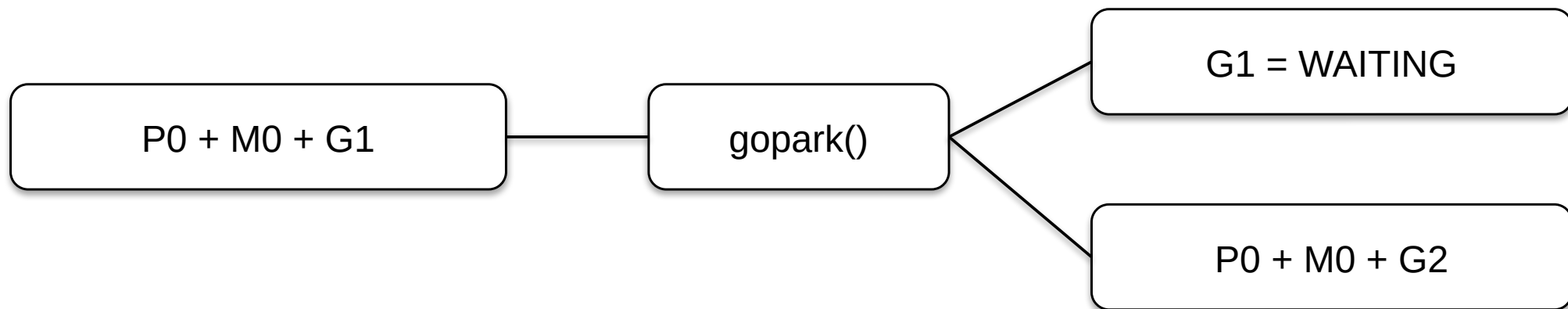
Go runtime synchronization internals

gopark / goready / sudog / semaphore / sync.Mutex

Главный вопрос

- Что происходит, когда `<-ch` или `mu.Lock()` прямо сейчас не могут продолжить выполнение?
- Почему ожидающая goroutine обычно не удерживает заблокированный OS thread?
- Как runtime запоминает waiter и как возвращает его обратно scheduler?

Блокируем G, а не M

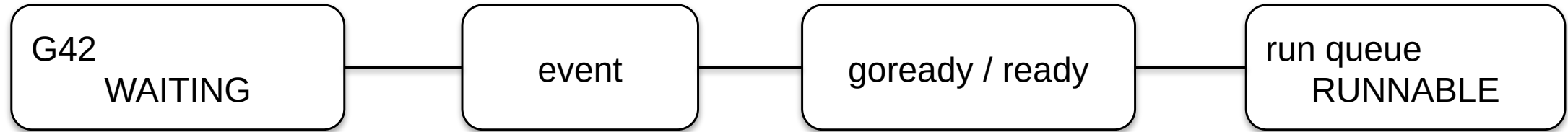


Ключ: ожидание scheduler-aware primitive паркует G; M и P остаются полезными для других runnable goroutines.

gopark: RUNNING → WAITING

- gopark означает: текущая goroutine сейчас не может продолжать работу.
- Runtime переключается на system stack g0, переводит G в _Gwaiting и отвязывает её от M.
- Текущий M возвращается в schedule() и получает другую runnable G.

goready: WAITING → RUNNABLE

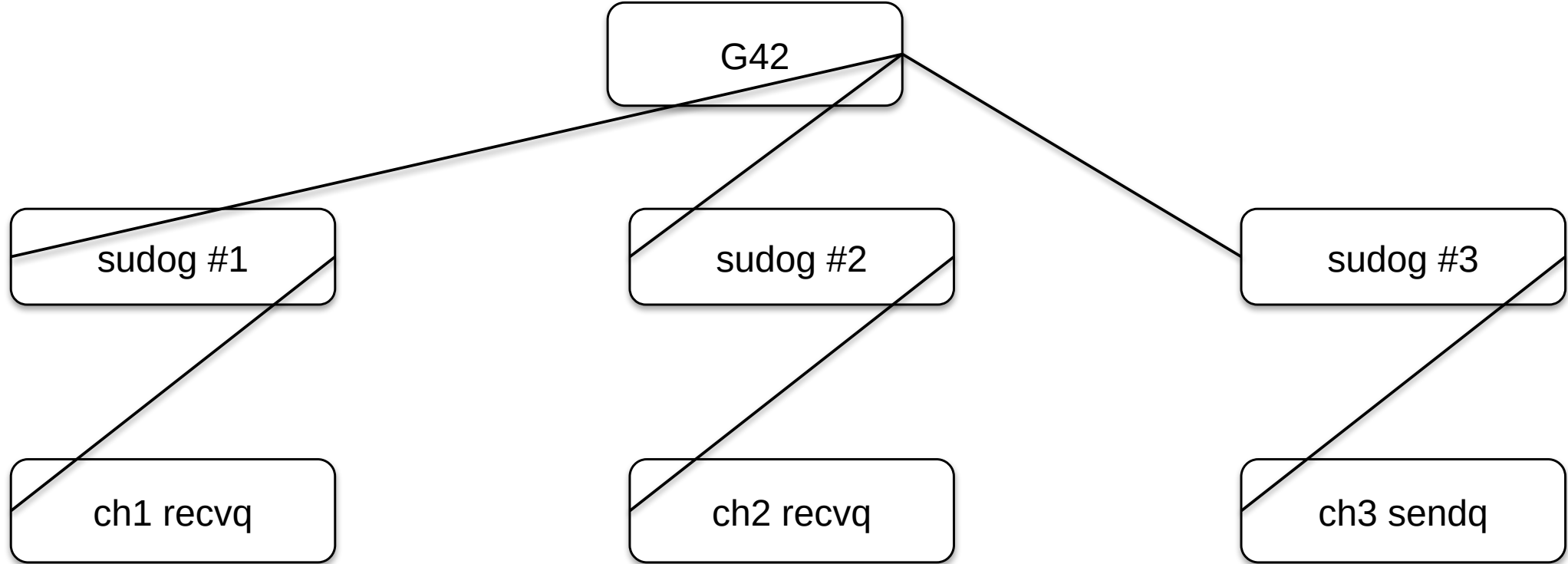


goready не означает «исполняй сейчас». Она только возвращает G в конкуренцию за P/M.

Что такое sudog

- sudog — runtime-запись goroutine в очереди ожидания конкретного synchronization object.
- Она связывает G с объектом ожидания: channel, semaphore и другими wait structures.
- sudog — не сама goroutine; это отдельный waiter node.

Почему не хранить просто *g

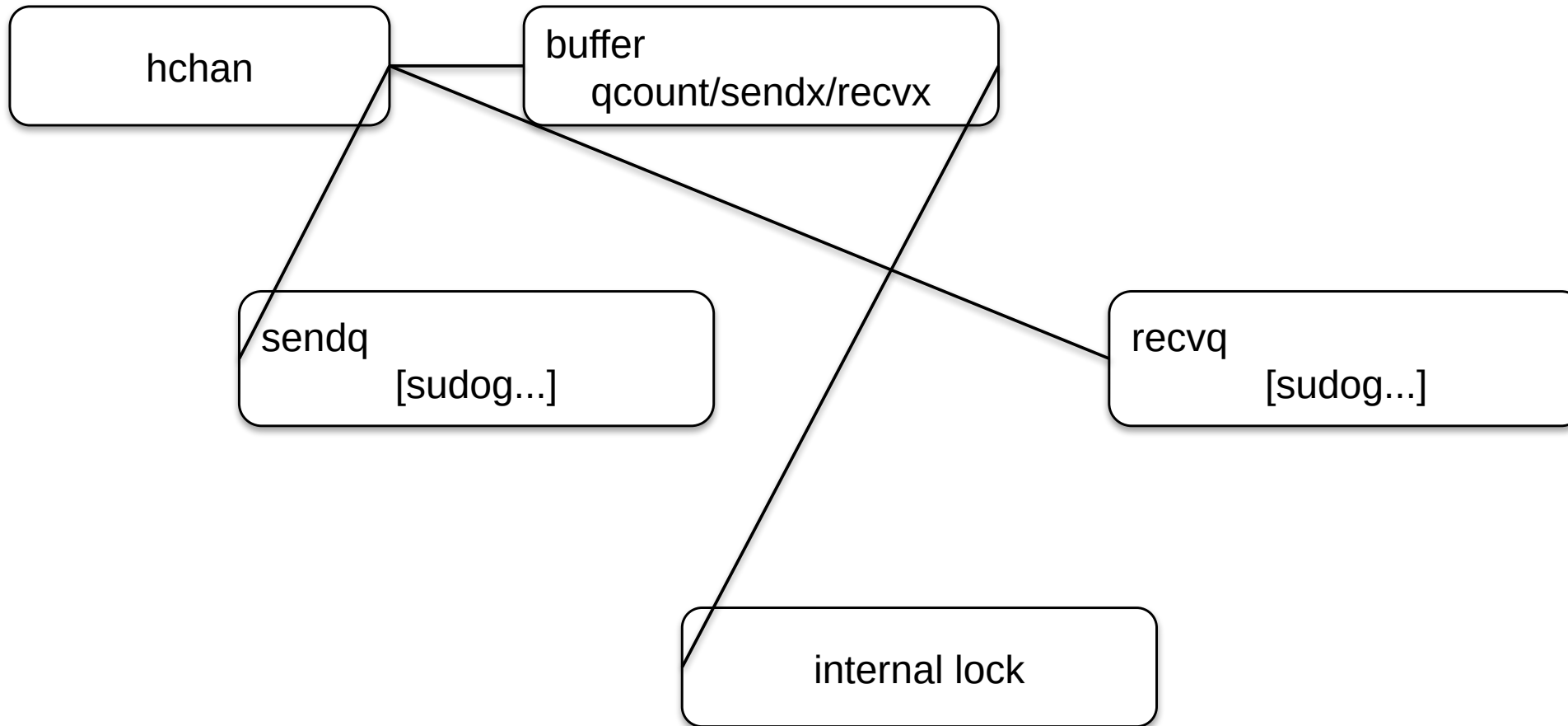


select может зарегистрировать одну G сразу в нескольких wait queues → нужна many-to-many связь через sudog.

sudog pool

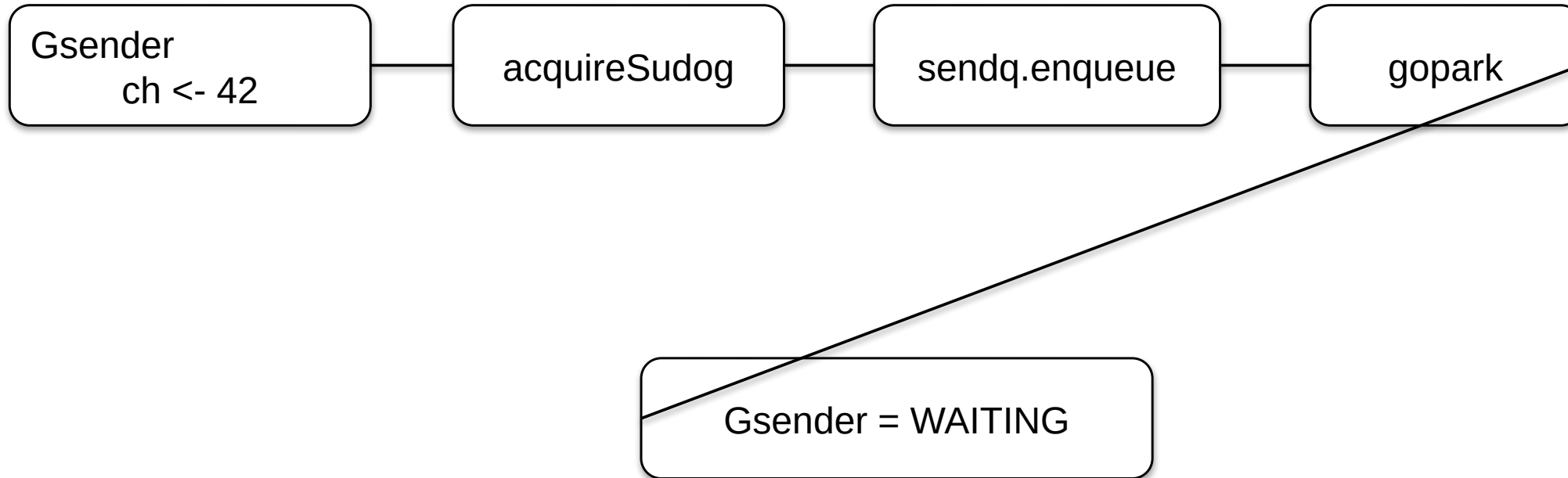
- Blocking operations очень частые; malloc/free sudog на каждое ожидание был бы дорогим.
- Runtime использует acquireSudog()/releaseSudog() и переиспользует waiter nodes.
- Это типичный Go runtime-подход: дешёвый fast path + специальные пулы для горячих внутренних объектов.

Channel: внутренние очереди ожидания



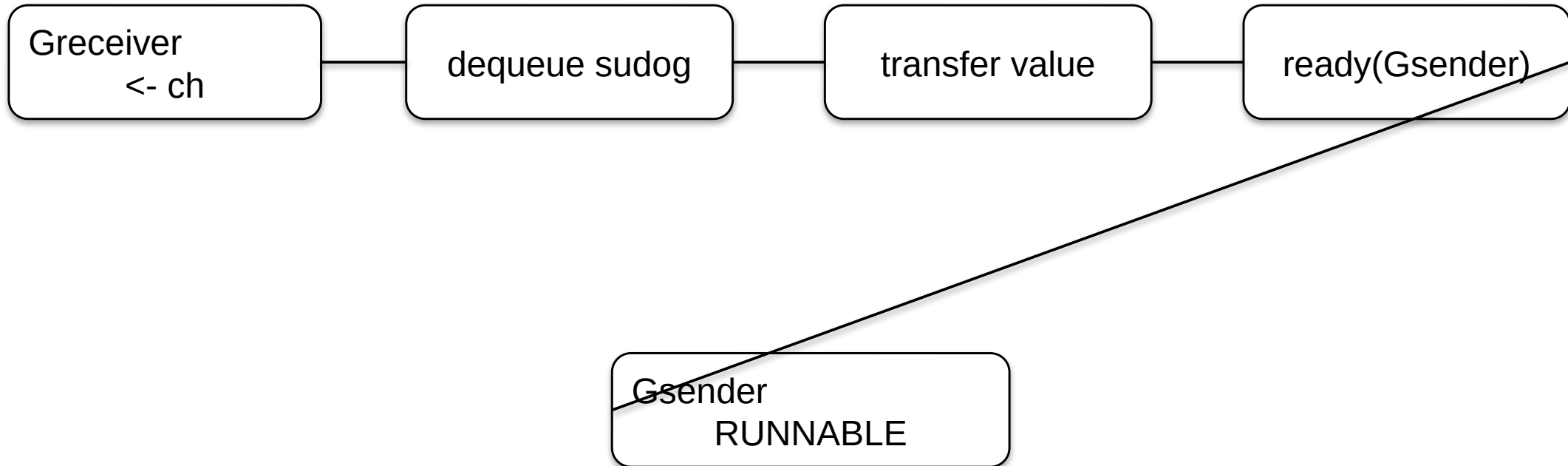
sendq хранит заблокированных sender, recvq — receiver. Lock защищает state machine channel, а не пользовательское ожидание целиком.

Unbuffered send: receiver отсутствует



sudog хранит ссылку на G и данные/адрес элемента, необходимые для rendezvous.

Receiver приходит позже



Пробуждённый sender снова идёт через scheduler; receiver не «вставляет» его прямо на CPU.

Lost wakeup: главный concurrency-риск

- Плохой алгоритм: check condition → context switch → signal без waiter → park.
- Если waiter ещё не зарегистрирован, сигнал может потеряться, а G заснуть навсегда.
- Поэтому register waiter, release internal lock и park должны образовывать согласованный protocol.

Зачем gorpark получает unlock callback

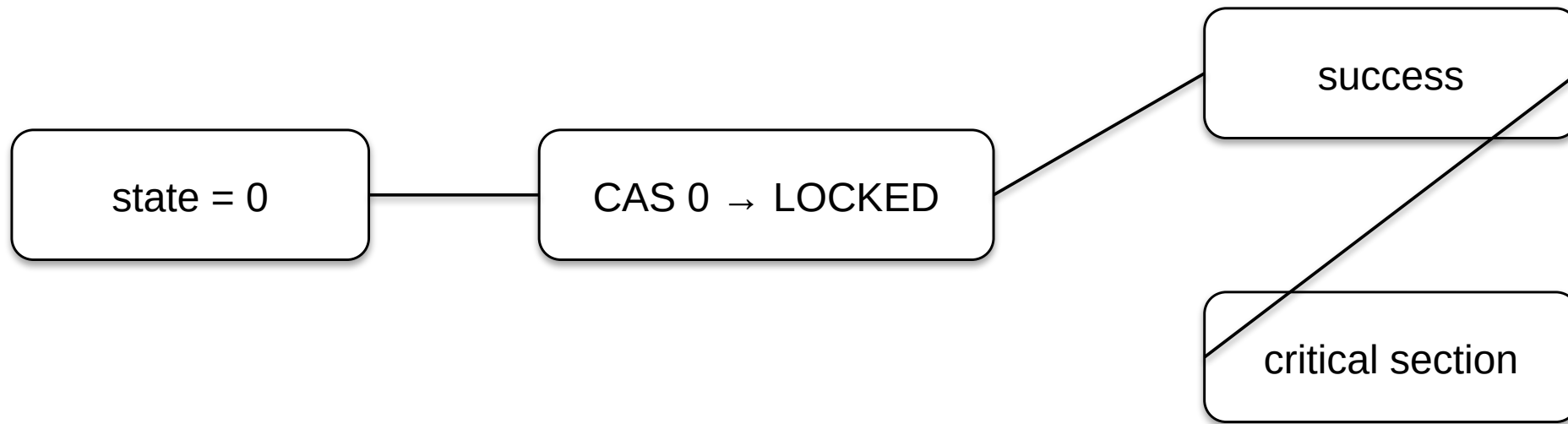
- Runtime должен выпустить internal lock в правильный момент — после регистрации waiter, но до сна.
- Так signal-side либо видит ещё незапаркованную G под lock, либо уже видит её в wait queue.
- Это защита от missed wakeup; тот же класс проблемы есть у condition variables/futex/semaphore.

sync.Mutex: маленькая структура, сложный protocol

```
type Mutex struct {  
    state int32  
    sema  uint32  
}
```

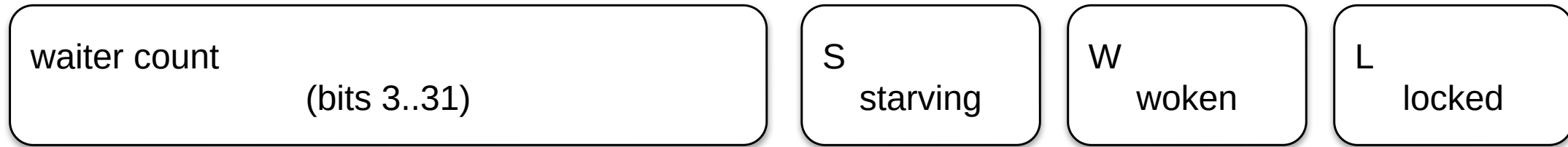
```
    // state: locked / woken / starving / waiter count
```

Mutex fast path: CAS



Uncontended mutex: atomics и return. Никаких sudog, semaphore, park и scheduler slow path.

Mutex state: packed bits

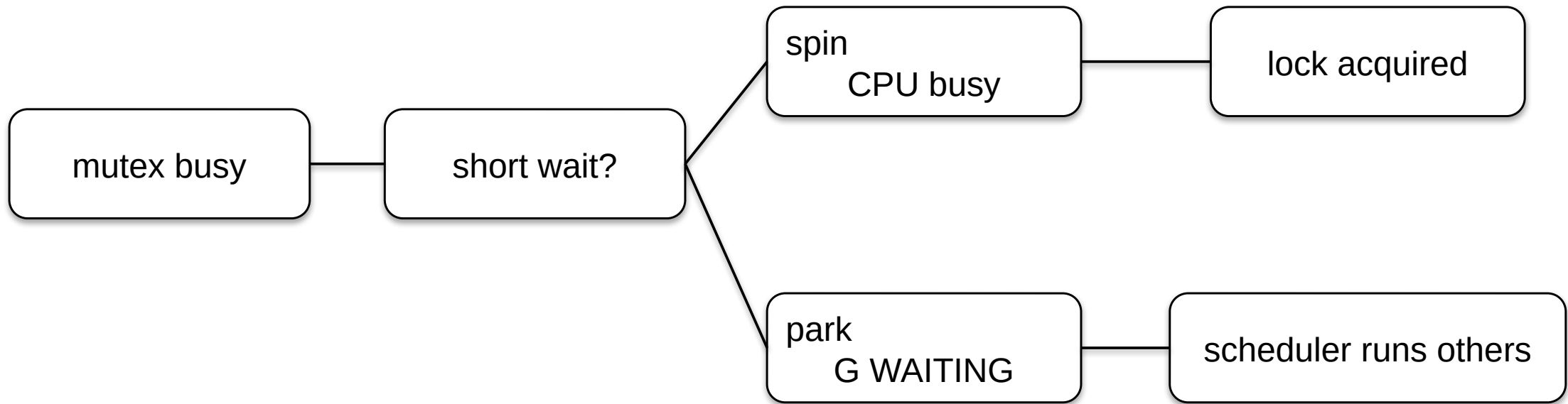


Один int32 хранит lock state, признак уже пробуждённого waiter, starvation mode и число ожидающих.

Lock slow path: зачем spinning

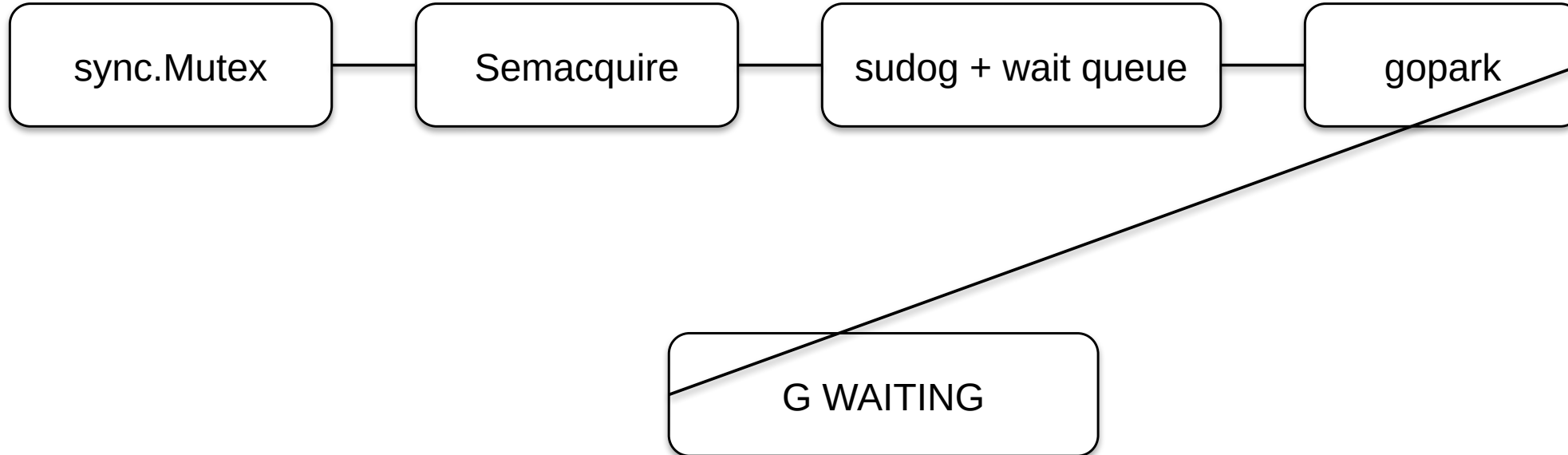
- CAS проигран: mutex занят. Но critical section может закончиться быстрее, чем стоит park/unpark.
- Runtime разрешает короткое active spinning: G остаётся RUNNING и проверяет lock ещё несколько раз.
- Spinning = обмен CPU на меньшую latency синхронизации.

Spinning vs parking



Runtime spin'ится консервативно: multicore, свободный смысл в spinning и отсутствие более полезной runnable работы на локальном P.

Runtime semaphore: мост к scheduler

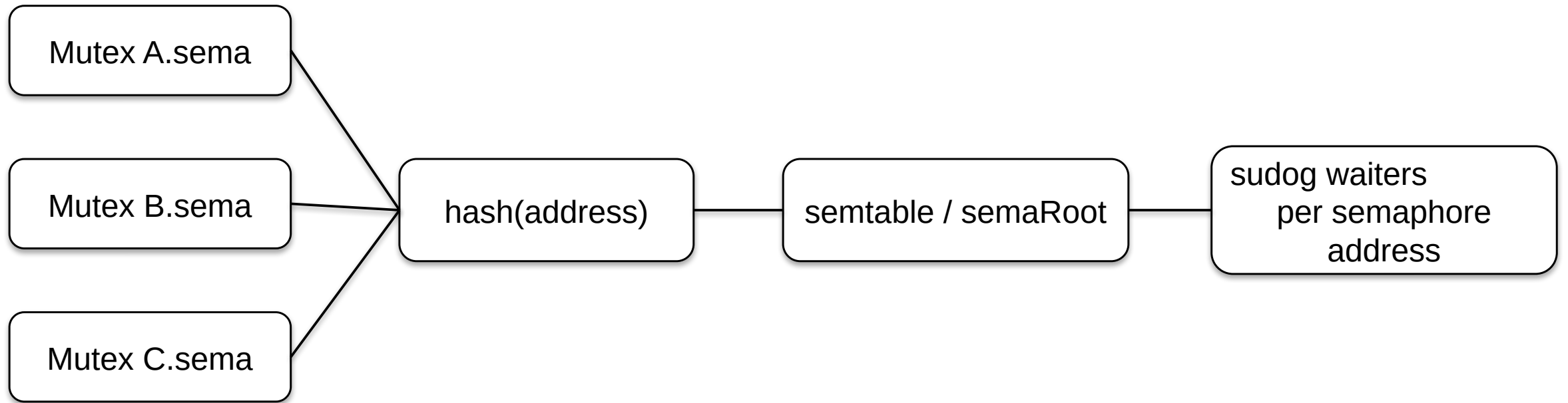


Это runtime semaphore, а не обязательный kernel semaphore: его задача — асинхронно park/ready goroutines.

semacquire1: почему condition проверяется повторно

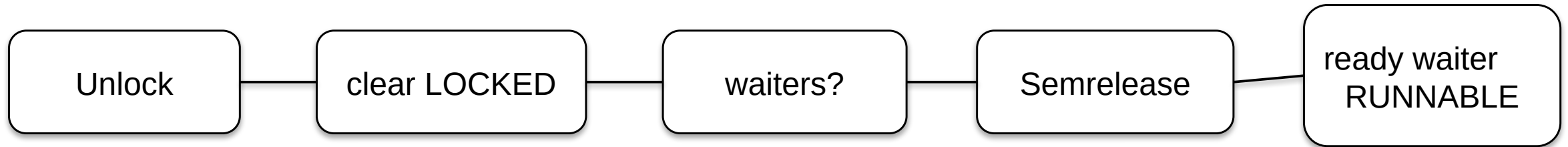
- Fast path пытается забрать semaphore token атомарно.
- Если token нет: увеличиваем waiter count, затем перепроверяем token перед enqueue/park.
- Повторная проверка закрывает окно гонки между первой проверкой и регистрацией waiter.

Где живут Mutex waiters



Очередь не лежит в каждом Mutex: сам mutex остаётся маленьким, тяжёлая wait-инфраструктура общая и включается только при contention.

Unlock → Semrelease → ready

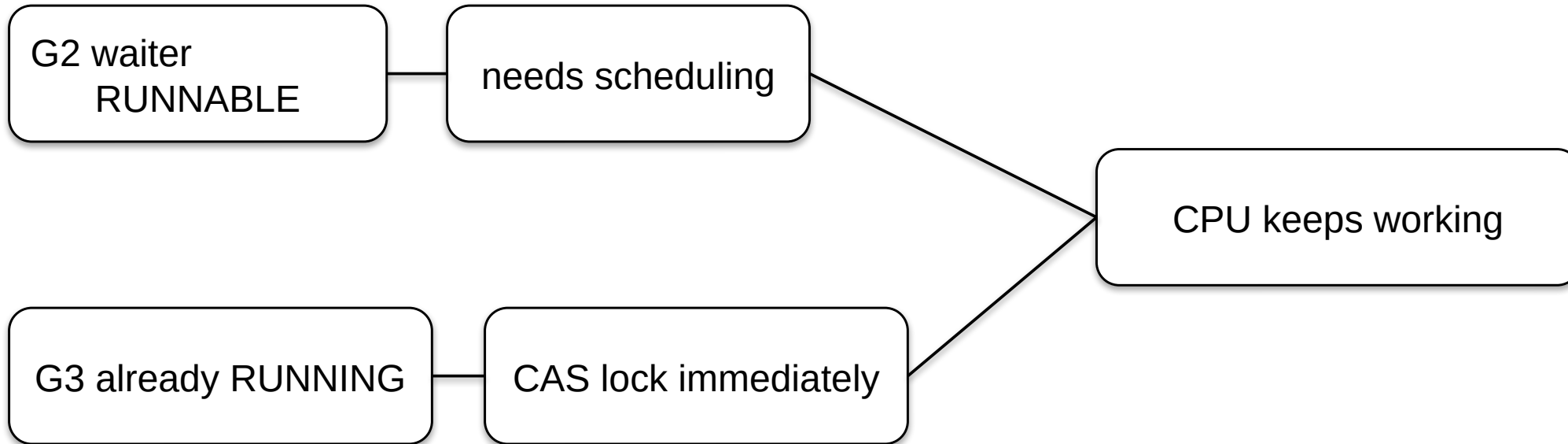


Fast path unlock может завершиться одной atomic операцией; scheduler подключается только если есть waiter/state для slow path.

Normal mode: barging

- Пробуждённый waiter ещё должен получить CPU и снова конкурировать за mutex.
- Новая goroutine уже RUNNING и может успеть захватить освободившийся lock раньше старого waiter.
- Это barging: throughput выше, но старый waiter потенциально проигрывает несколько раз подряд.

Почему barging выгоден throughput



Normal mode предпочитает already-running contender → меньше handoff overhead и выше throughput.

Starvation mode

- Если waiter слишком долго проигрывает, текущая реализация может переключить mutex в starvation mode.
- В starvation mode ownership фактически передаётся следующему waiter; newcomers не должны steal lock.
- Trade-off: fairness и p99 лучше, throughput хуже. Порог порядка 1 ms — implementation detail, не API contract.

Normal vs starvation

NORMAL

Unlock

wake waiter

newcomer may
steal

STARVATION

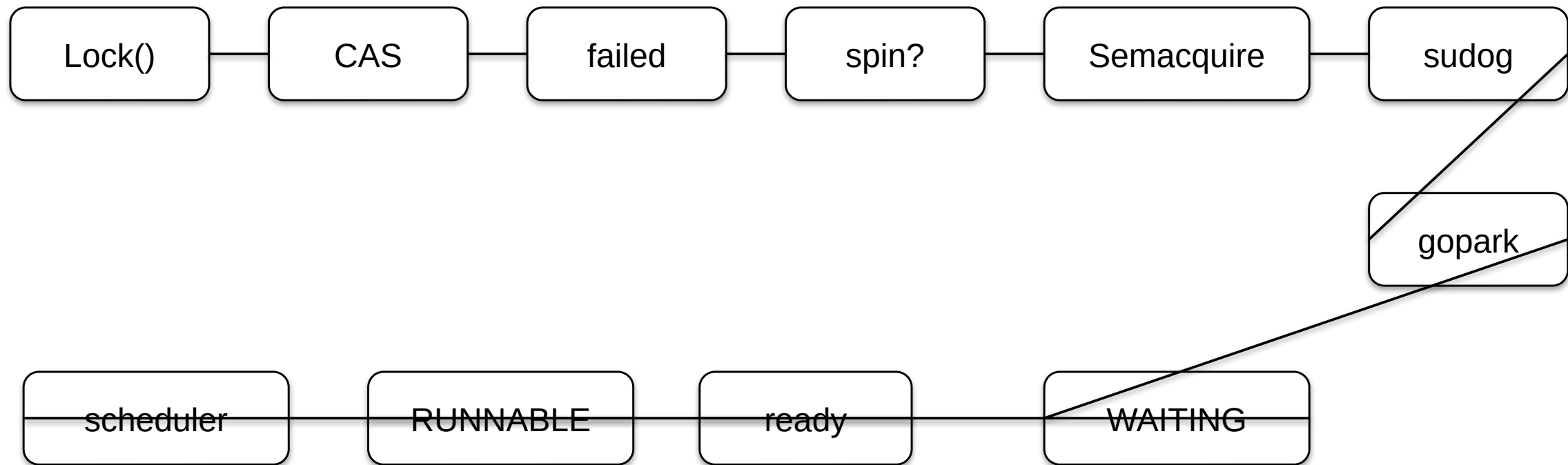
Unlock

direct handoff

newcomer queues

Normal optimizes throughput. Starvation mode optimizes fairness/tail latency.

Полный Mutex.Lock slow path



Fast path заканчивается на CAS. Весь остальной runtime-механизм появляется только при contention.

Channel и Mutex: один runtime-паттерн

- Channel: wait queues sendq/recvq находятся внутри hchan.
- Mutex: waiters обслуживаются runtime semaphore table.
- Общая идея одинакова: register waiter → sudog → gopark → event → ready/goready → RUNNABLE.

Что происходит под contention

- Много goroutines могут быть WAITING на одном mutex при низкой загрузке CPU.
- Это serialization bottleneck, а не scheduler saturation.
- Рост contention добавляет spinning, park/ready, queueing и scheduler latency вокруг самой критической секции.

Backend-пример: глобальный cache mutex

- На TEST 10 RPS: Lock → map lookup → Unlock выглядит бесплатным.
- На PROD 50 000 RPS: тысячи G конкурируют за одну critical section.
- Симптомы: p99 растёт, block/mutex profile показывает hotspot, CPU может быть далёк от 100%.

Диагностика

- Mutex profile: где goroutines теряют время в contested locks.
- Block profile: channel/mutex/Cond/select blocking.
- go tool trace: RUNNING → WAITING(sync.Mutex.Lock) → RUNNABLE → RUNNING.
- runtime.NumGoroutine сам по себе недостаточен: нужен wait reason и время ожидания.

Практика

- Benchmark uncontended Mutex: доказать дешевизну CAS fast path.
- RunParallel + разная длительность critical section: увидеть переход spin → park.
- Unbuffered channel: через trace увидеть sendq/recvq park/ready.
- 100 000 goroutines ждут channel: сравнить goroutine count и OS thread count.

Что важно запомнить

- gopark паркует G; goready/ready возвращает её в RUNNABLE.
- sudog — waiter node, связывающий G с synchronization object.
- Mutex fast path — atomics; slow path — spinning → semaphore → parking.
- Semaphore wait queues позволяют заблокировать goroutine без отдельного blocked OS thread.
- Normal/starvation modes балансируют throughput и fairness.
- Под нагрузкой проблема часто не в mutex как инструкции, а в очереди перед critical section.

sync.Mutex.state и lockSlow()

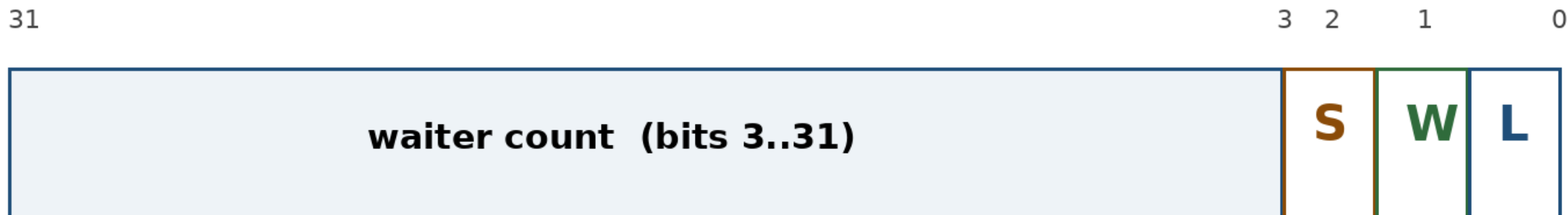
- Цель: понять Mutex как адаптивный runtime-протокол, а не как две функции Lock/Unlock.
- Путь: state bits → CAS fast path → spin → waiter accounting → semaphore → starvation handoff.
- Все конкретные биты и пороги — детали текущей реализации Go runtime.

Базовая структура Mutex

```
type Mutex struct {  
    state int32  
    sema  uint32  
}
```

Вся логика contention упакована в 32-битное state и runtime semaphore.

Mutex.state: 32 бита одной state machine



L lock занят обычным владельцем

W один активный contender уже назначен на пробуждение

S mutex в starvation mode; очередь получает приоритет

waiters количество sleeping/зарегистрированных waiters = $\text{state} \gg 3$

Константы state

```
const (  
    mutexLocked = 1 << iota  
    mutexWoken  
    mutexStarving  
    mutexWaiterShift = iota  
)
```

После трёх флагов остальные биты кодируют waiter count.

Как читать state

Примеры

- `state = 0`` → свободен
- `state = 1`` → Locked
- `state = 9`` → Locked + 1 waiter
- `state = 11`` → Locked + Woken + 1 waiter

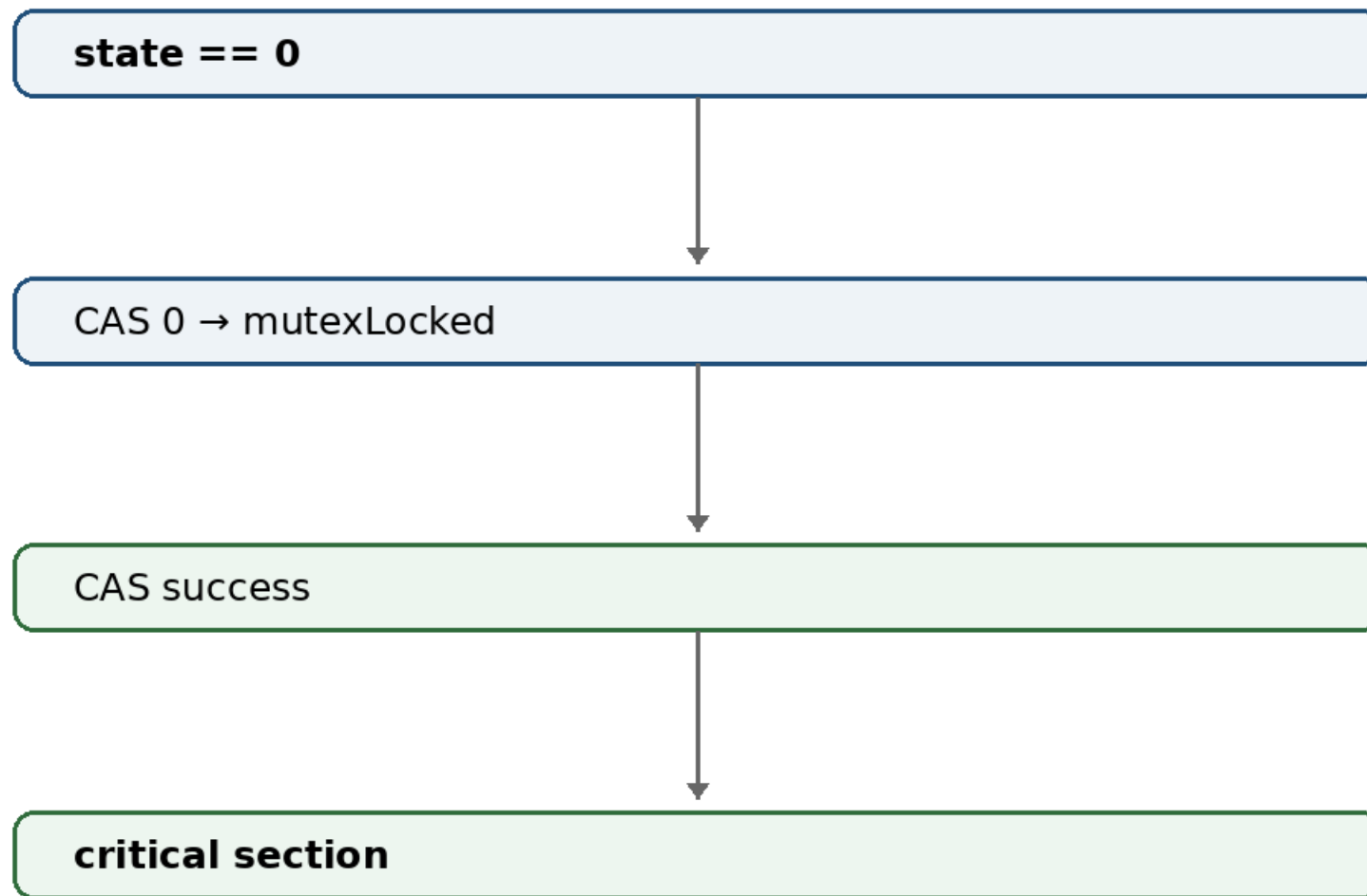
Формула

- `waiters = state >> 3`
- `+8`` означает ещё одного waiter
- младшие три бита — режим/координация

Зачем всё в одном `int32`?

- Связанные признаки изменяются одним CAS: `old state` → `new state`.
- Никакого отдельного `lock` для самого `Mutex.state` не требуется.
- Это компактная `state machine`: прочитал снимок → вычислил новый → попытался CAS → при конфликте пересчитал.

Lock(): common case должен быть почти бесплатным



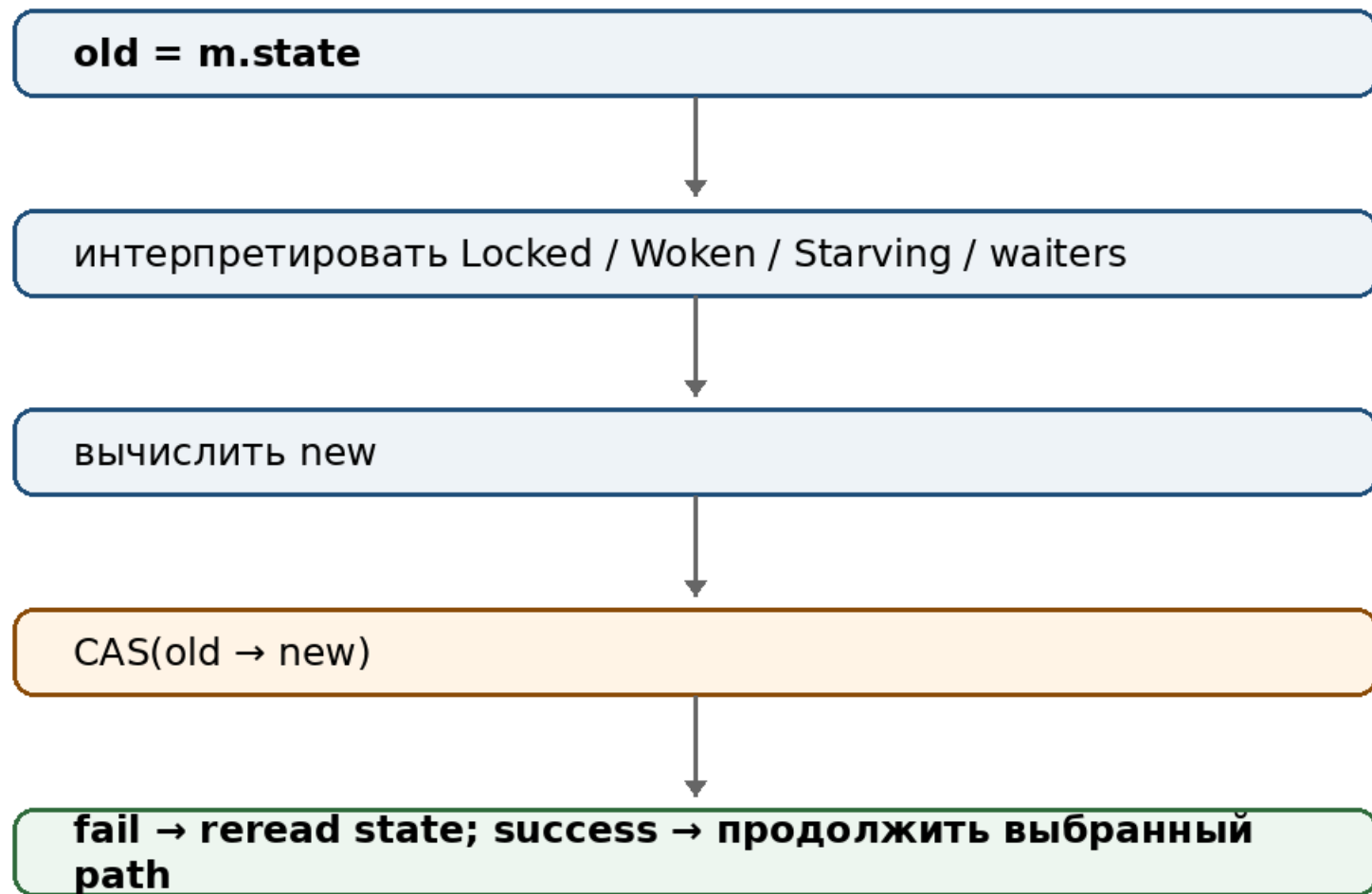
Uncontended path не идёт в scheduler, semaphore и sudog.

Fast path

```
if atomic.CompareAndSwapInt32(  
    &m.state,  
    0,  
    mutexLocked,  
) {  
    return  
}  
  
m.lockSlow()
```

Вся сложность вынесена из common path в lockSlow().

lockSlow(): центральная CAS state machine



Локальные переменные lockSlow()

- ``waitStartTime`` — когда началось реальное ожидание.
- ``starving`` — эта конкретная G считает ожидание слишком долгим.
- ``awoke`` — эта попытка Lock уже участвует как активный пробуждённый contender.
- ``iter`` — сколько раз мы пытались active spin.

mutexWoken: зачем отдельный bit?

- Mutex может иметь sleeping waiters и одновременно уже активного contender.
- `mutexWoken=1` говорит Unlock: кандидат уже есть, лишнего waiter будить не надо.
- Это уменьшает park → wake → schedule → проигрыш CAS → park churn.

Без mutexWoken: thundering herd



Одно освобождение ресурса не требует пробуждать толпу.

awoke vs mutexWoken

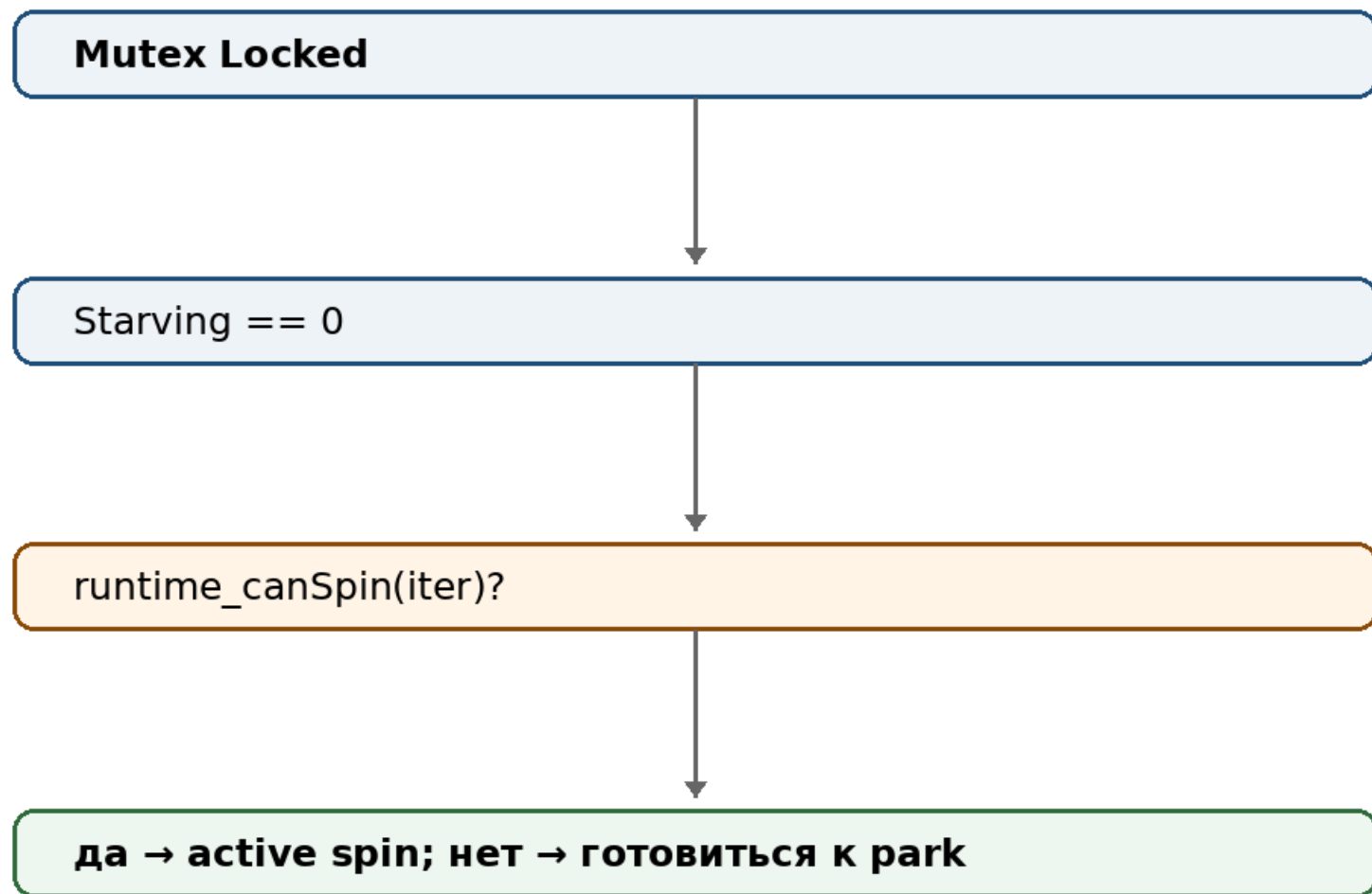
awoke

- локальная переменная одного Lock()
- значит: именно эта G уже активный кандидат
- позже эта G должна корректно снять Woken bit

mutexWoken

- глобальный bit в m.state
- координирует Unlock и contenders
- предотвращает лишнее пробуждение

Первый выбор lockSlow: можно ли spin?



Что такое active spinning

- G остаётся RUNNING и несколько раз ждёт освобождения lock, не отдавая P.
- Цена: M/P/CPU заняты почти бесполезной работой.
- Выгода: для очень короткой critical section park/unpark может быть дороже самого ожидания.

Когда spin разумен

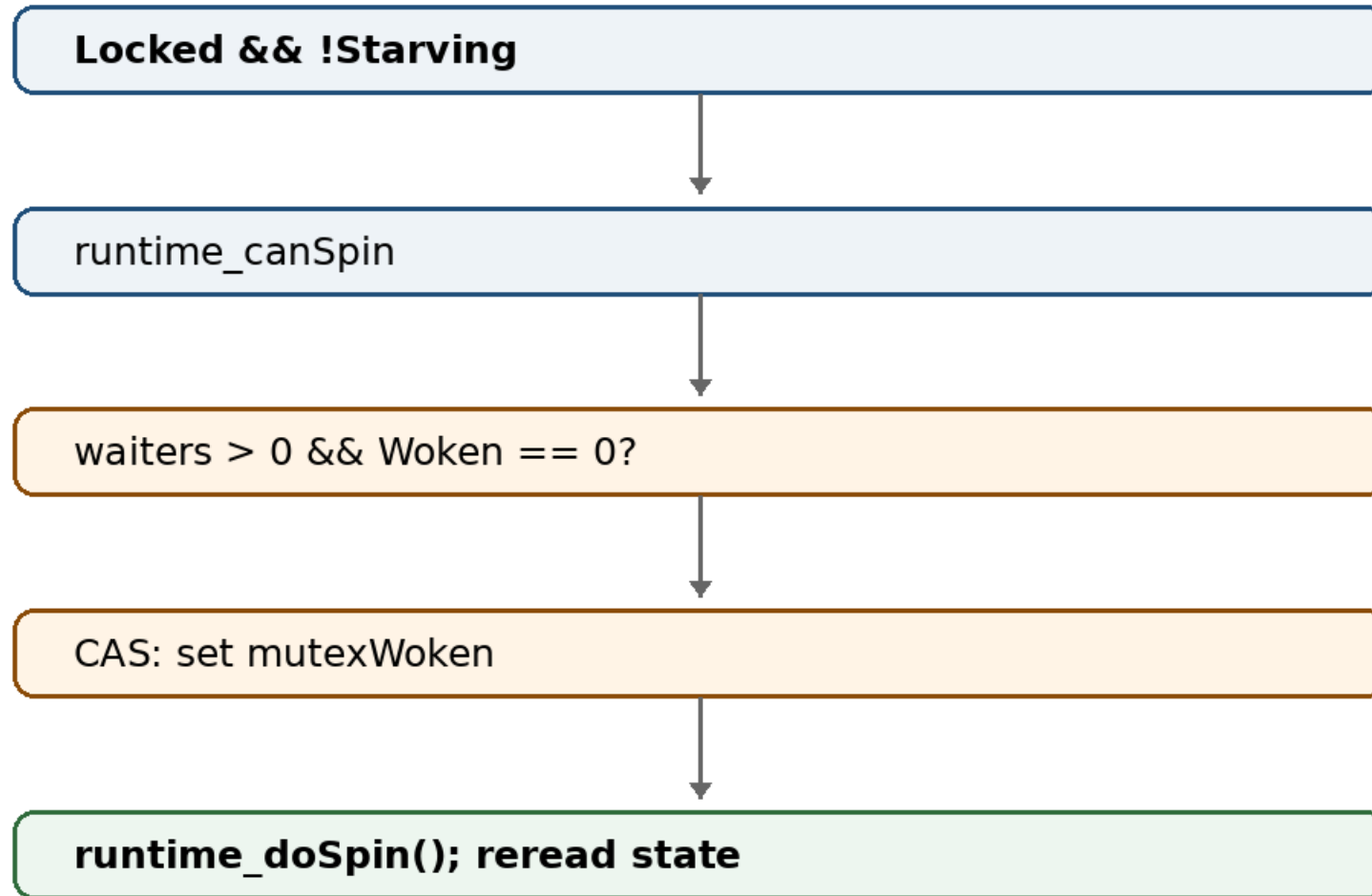
Хороший случай

- несколько CPU/P
- другая G реально может освободить lock
- локальная run queue текущего P пуста
- ожидание ожидается коротким

Плохой случай

- GOMAXPROCS=1
- есть runnable work
- critical section миллисекунды
- starvation mode уже активен

Spin path и mutexWoken



После spin: строим new state

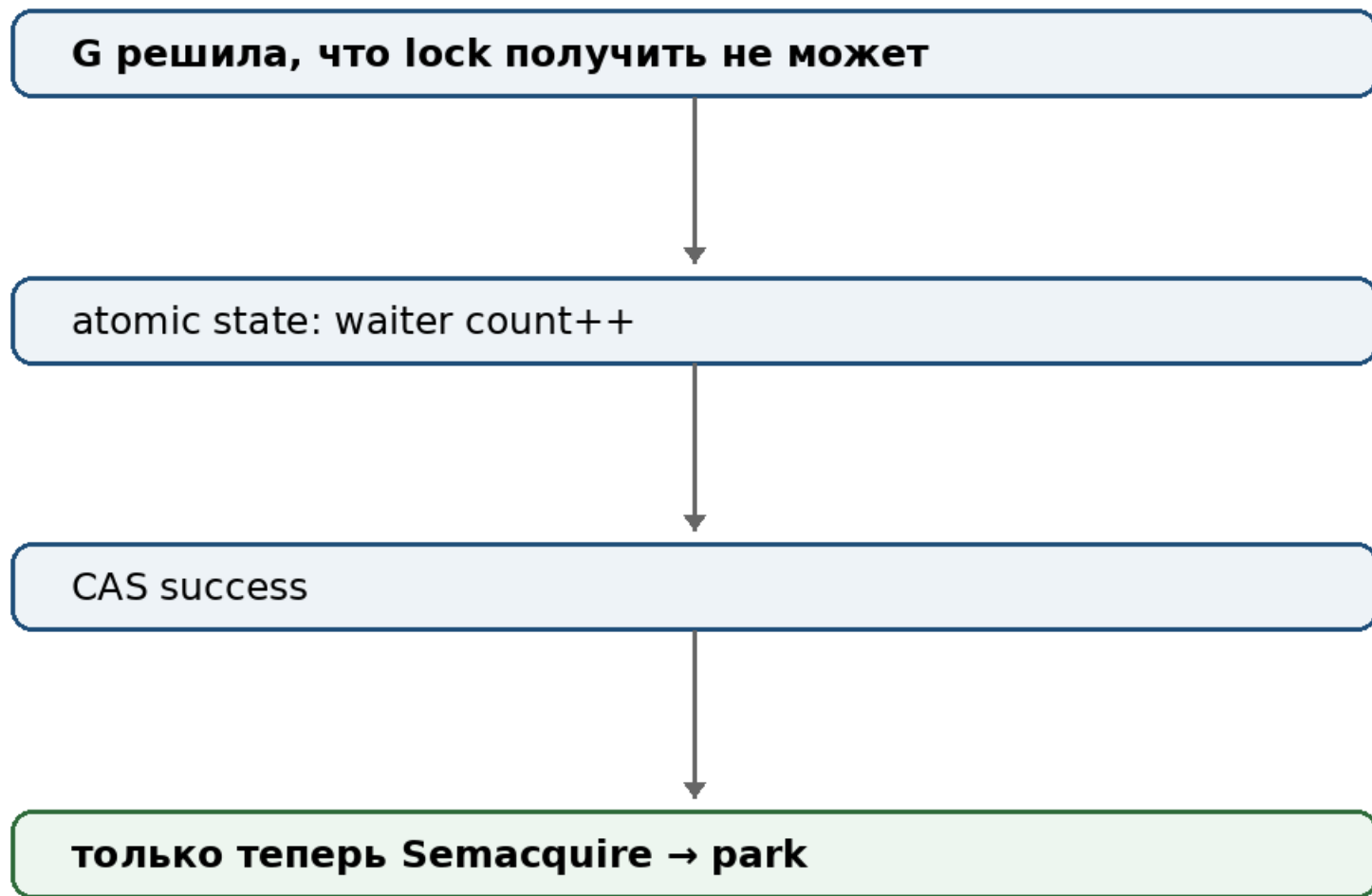
- Если normal mode — текущая G может запросить ``mutexLocked``.
- Если mutex занят или starving — ``waiter count++`` (``+ 1 << 3``).
- Если G ждала слишком долго — готовим ``mutexStarving``.
- Если ``awoke=true`` — текущая попытка должна снять глобальный ``mutexWoken``.

Waiter count — это просто старшие биты

```
new += 1 << mutexWaiterShift  
  
// mutexWaiterShift == 3  
// => +8 к state = +1 waiter
```

Например: Locked + 2 waiters = 1 + 16 = 17.

Почему waiter++ происходит до sleep



Иначе Unlock может увидеть waiters=0 и потерять wakeup.

Starvation: когда throughput начинает ломать fairness

- В normal mode awakened waiter ещё конкурирует с newly arriving goroutines.
- Новая G уже RUNNING и может выиграть CAS раньше старого waiter — barging.
- Если один waiter проигрывает слишком долго, runtime переключает mutex в starvation mode.
- Текущий implementation threshold — порядка 1 ms; это не API guarantee.

Normal mode vs starvation mode

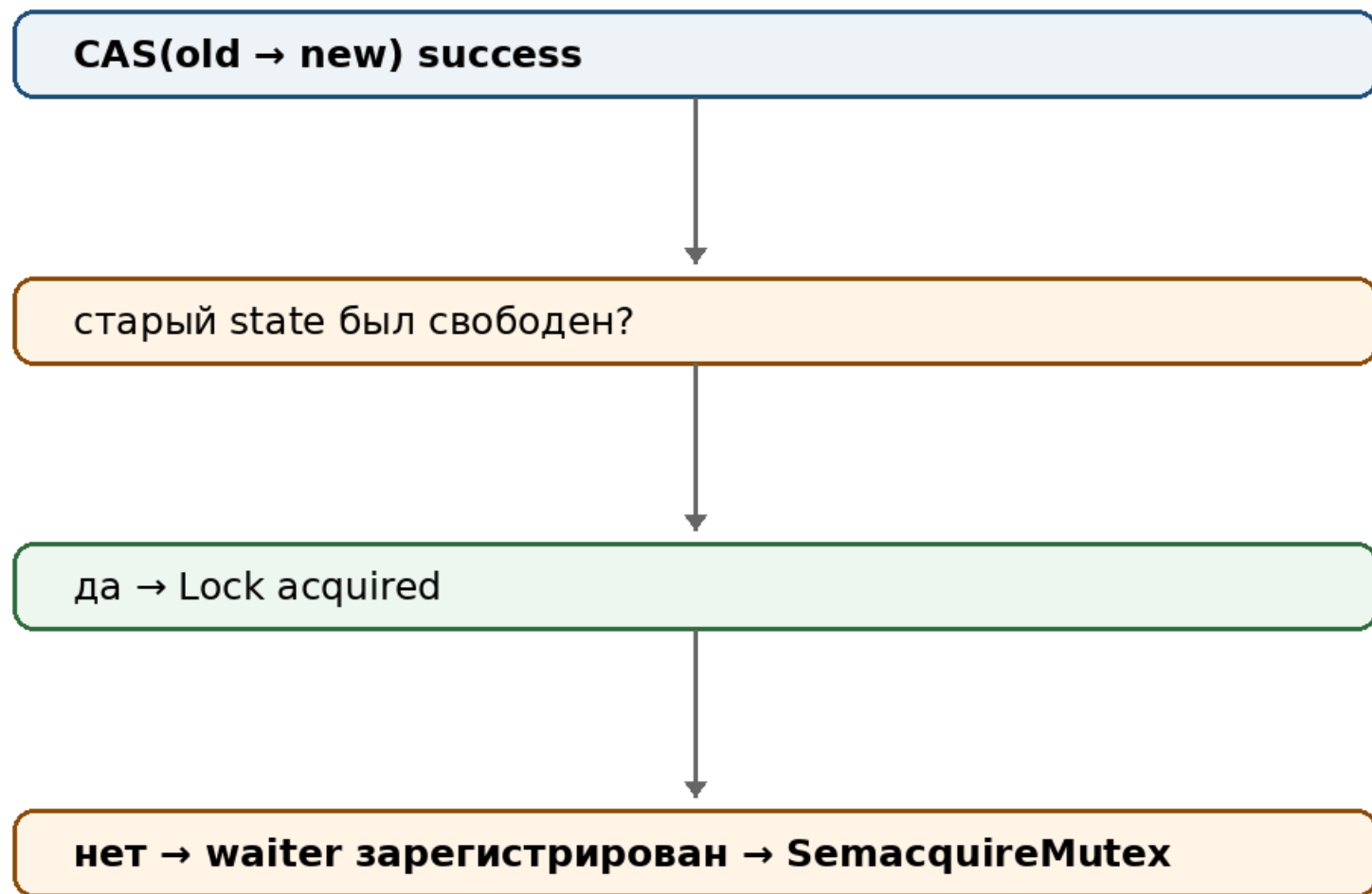
Normal

- Unlock будит waiter
- waiter снова конкурирует
- newcomer может украсть lock
- лучше throughput

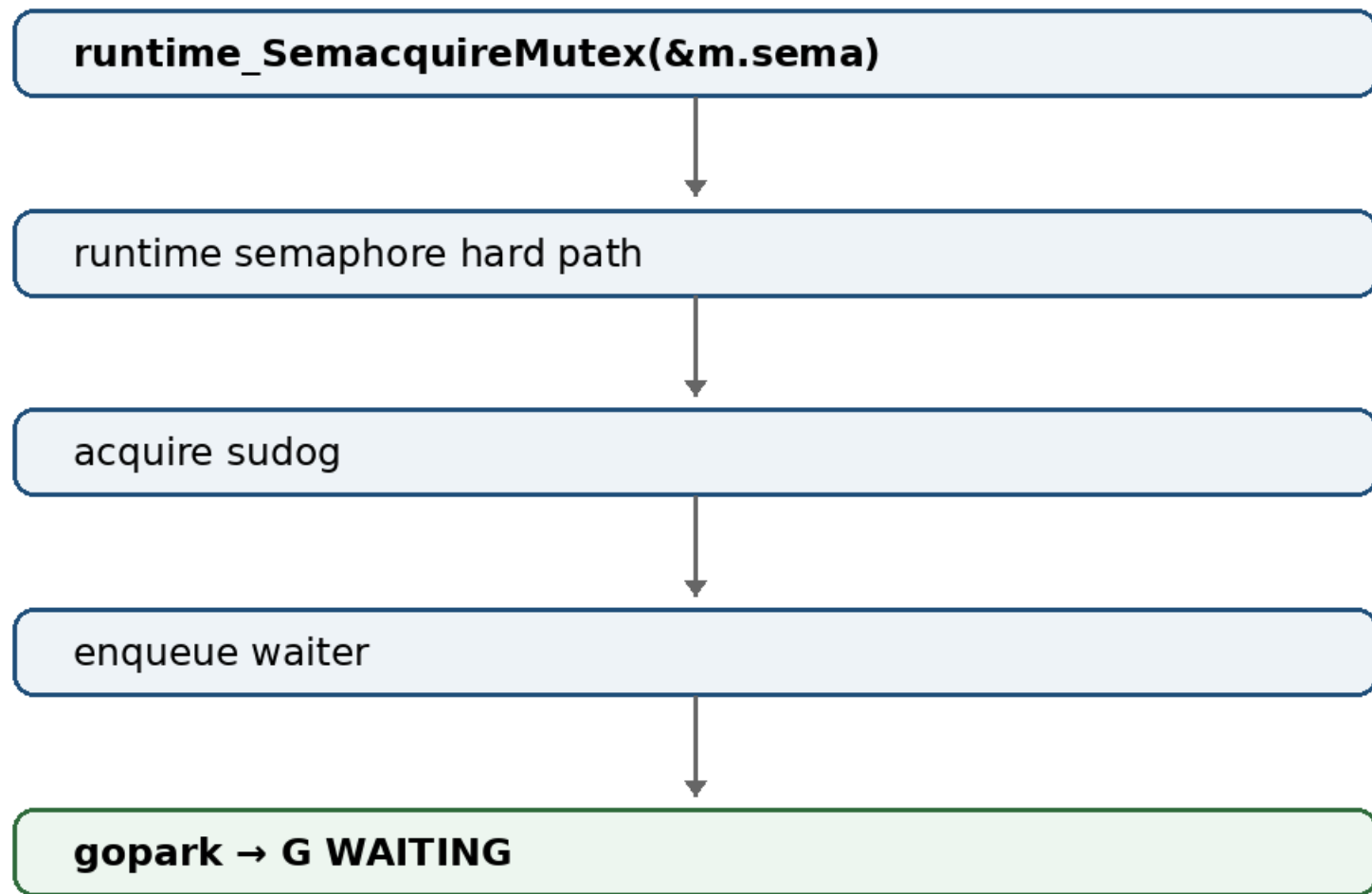
Starvation

- очередь получает приоритет
- newcomers не spin и не steal
- ownership передаётся следующему waiter
- лучше fairness / tail latency

CAS завершился успешно: два сценария



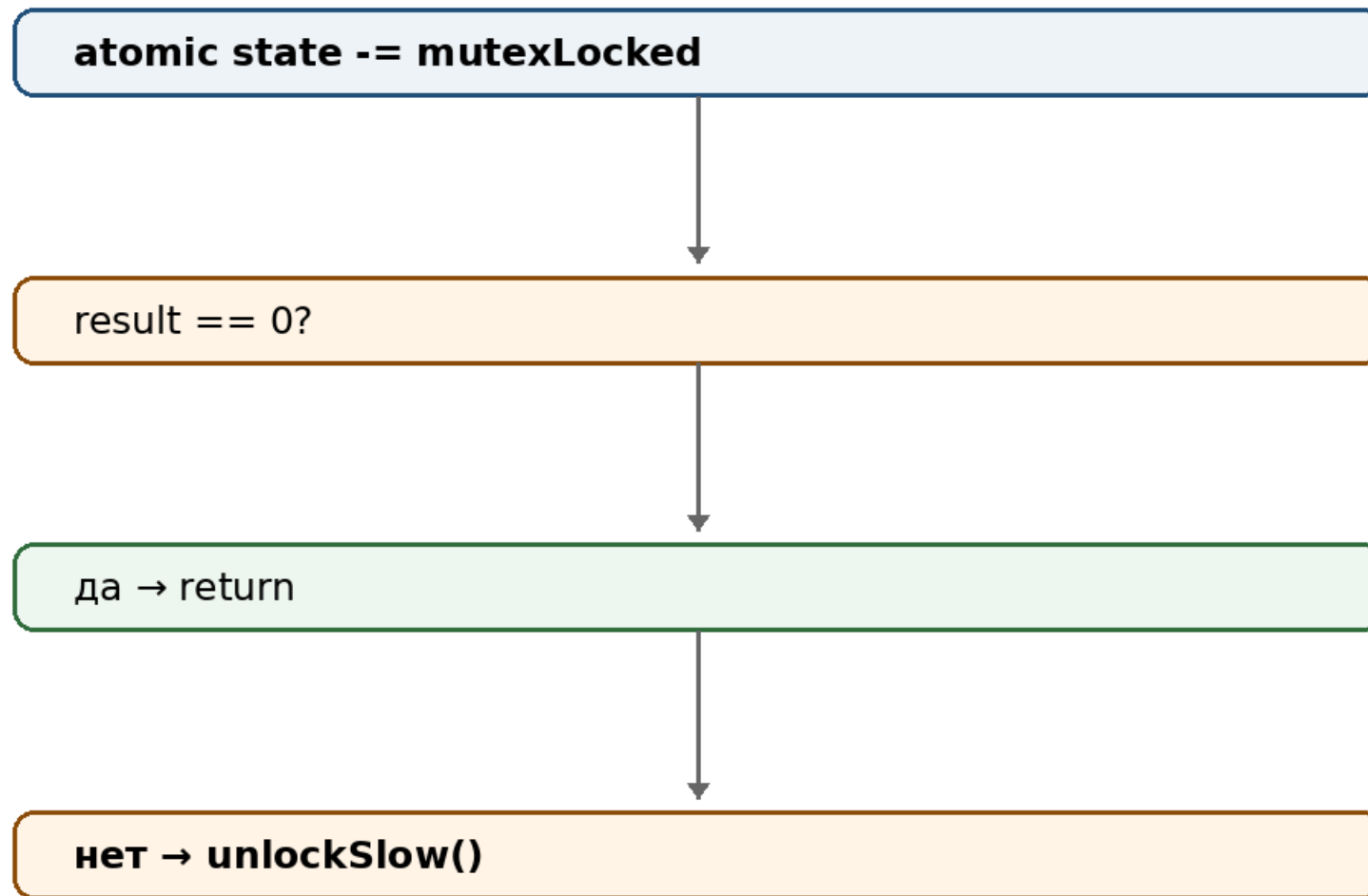
Slow waiting path



Что происходит с М/Р во время ожидания

- Ожидающая G переходит RUNNING → WAITING.
- М больше не обязан быть связан с этой G.
- Scheduler может выполнить на том же Р другую runnable goroutine.
- Поэтому contended Go Mutex обычно блокирует goroutine, а не выделенный OS thread.

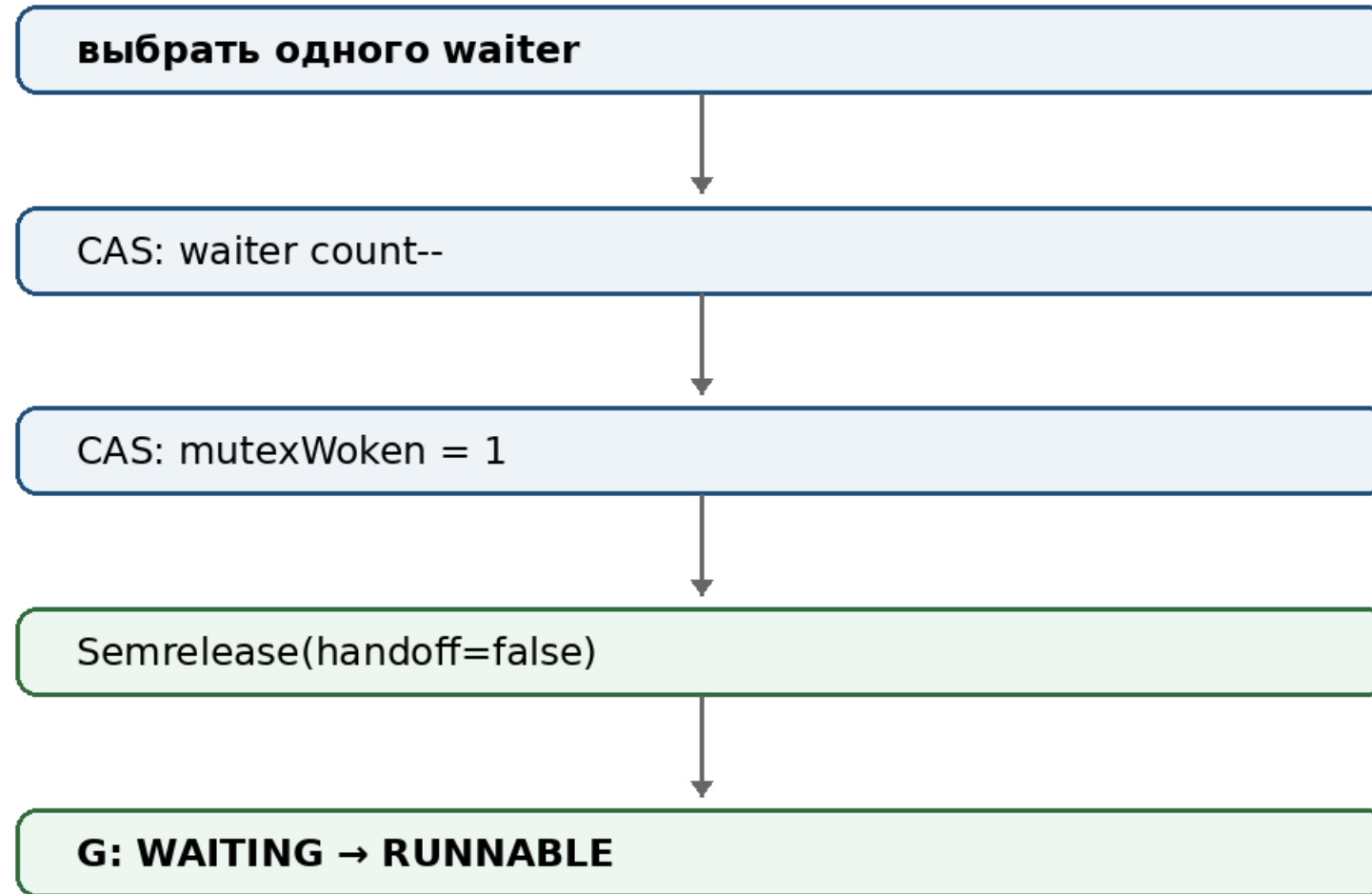
Unlock(): fast path тоже очень короткий



Normal unlockSlow: когда никого не будим

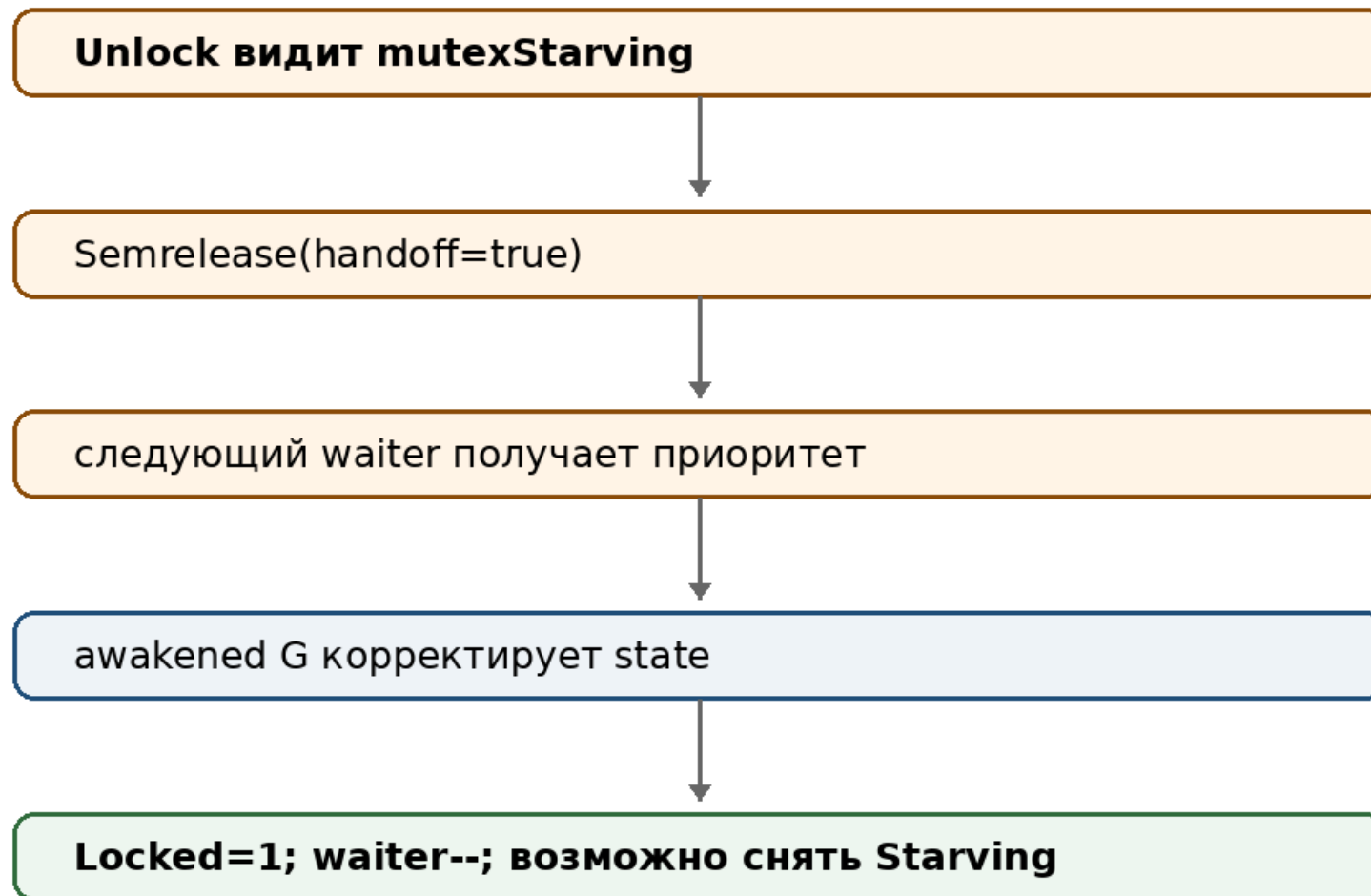
- `waiters == 0` — будить некого.
- `mutex` уже снова `Locked` — другой contender успел захватить его.
- `mutexWoken == 1` — активный candidate уже существует.
- `Starving == 1` — работает другой handoff protocol.

Normal wakeup protocol



Сначала фиксируем state, потом фактически будим G.

Starvation handoff

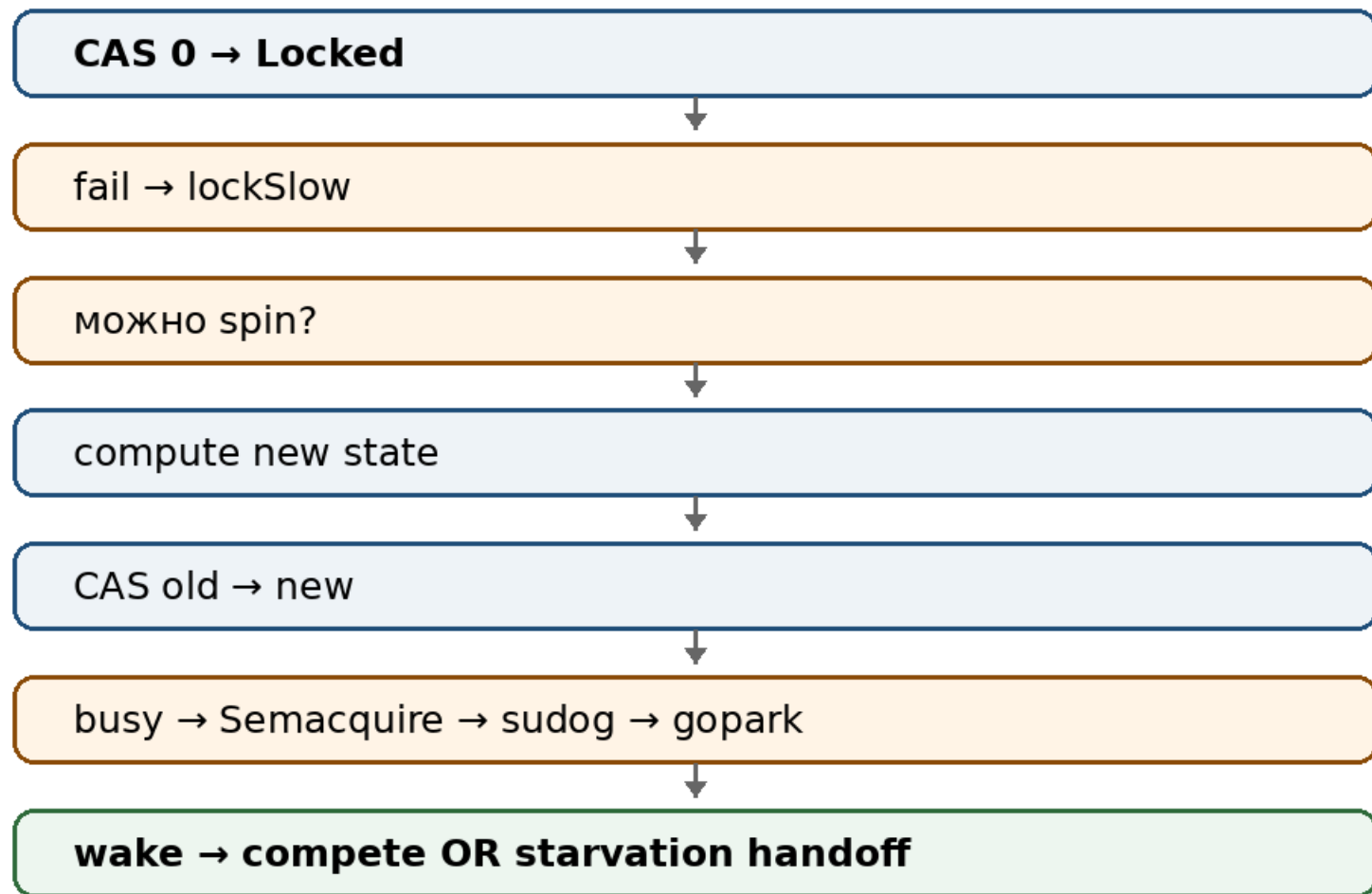


Почему в starvation встречается Locked = 0

```
state:
    mutexStarving = 1
    mutexLocked   = 0
    waiters       > 0
```

Locked=0 здесь не означает свободный Mutex: starvation bit резервирует ownership за очередью.

Полный Lock pipeline



Что меняется под нагрузкой

Low contention

- почти все Lock = один CAS
- минимум scheduler activity
- Mutex выглядит «бесплатным»

High contention

- CAS failures
- active spinning
- waiter queues
- park/wake
- serialization и p99 latency

Backend-антипаттерн: держать Mutex вокруг I/O

- `Lock → DB/HTTP/disk/sleep → Unlock` превращает один медленный downstream в очередь всех contenders.
- Симптом: CPU может быть невысоким, а latency огромной.
- Причина — serialization bottleneck и park/wake churn, а не «медленный CAS».

Диагностика

- Mutex profile — где время теряется на contended locks.
- Block profile — где goroutines реально блокируются.
- go tool trace — RUNNING → WAITING(sync.Mutex) → RUNNABLE → RUNNING.
- CPU profile — отделяет spinning/полезную работу от ожидания.
- Главный вопрос: сколько времени lock держат и что делают внутри critical section?

Что важно запомнить

- ``state`` — packed atomic state machine.
- Fast path = CAS; slow path = адаптивный протокол.
- ``mutexWoken`` ограничивает лишние wakeups.
- waiter count регистрируется до park — защита от lost wakeup.
- spinning меняет CPU на меньшую park/unpark latency.
- normal mode оптимизирует throughput; starvation mode — fairness.
- Под нагрузкой цена Mutex — очередь и serialization, а не сам atomic instruction.